Engineering Manager iOS

Complete Career Roadmap & Interview Preparation Guide

Swift & Objective-C · UIKit & SwiftUI · Architecture Patterns · Swift Concurrency · Performance Optimization · CI/CD & DevOps · People Management · Hiring & Onboarding · System Design · STAR Interview Stories · 30-60-90 Day Plan · Skill-Building Roadmap

6 Documents · 3,700+ Lines · Basic to Advanced · Apple · Google · Meta · Amazon

Engineering Manager - iOS: Complete Interview Preparation & Career Roadmap

> *A comprehensive, structured guide covering everything you need to know — from fundamentals to advanced leadership — to land an Engineering Manager (iOS) role at top-tier tech companies.*

Table of Contents

1. Role Overview & Expectations

What Is an Engineering Manager - iOS?

An Engineering Manager (EM) for iOS is a hybrid leader who:
- Manages a team of iOS engineers (typically 4–12 direct reports)
- Is accountable for the technical quality, delivery, and growth of the iOS platform/product
- Acts as a bridge between product, design, data, and engineering
- Sets technical direction while staying close to the code (in most companies)
- Hires, retains, and grows engineers

Difference: EM vs. Tech Lead vs. Staff Engineer

Dimension	Tech Lead	Staff Engineer	Engineering Manager
Primary focus	Technical execution	Technical strategy	People + delivery
Reports	None (usually)	None	4–12 engineers
Coding	50–80%	30–70%	0–30%
Career ownership	Self	Self + influence	Others' careers
Accountability	Code quality	System architecture	Team outcomes

Typical Responsibilities

- **People**: 1:1s, performance reviews, promotions, hiring, firing, coaching
- **Delivery**: Sprint planning, roadmap execution, removing blockers
- **Technical**: Architecture reviews, technical decisions, code quality standards
- **Strategy**: Long-term iOS platform vision, tech debt management
- **Culture**: Team health, psychological safety, engineering culture
- **Stakeholder management**: Sync with PM, Design, Data, QA, Leadership

What Top Companies Look For

Company	Key EM Traits
Apple	Deep platform expertise, quality obsession, confidentiality discipline
Google	Structured thinking, data-driven decisions, career development focus
Meta	Speed, impact at scale, cross-functional leadership
Airbnb	Craft, reliability, user empathy
Spotify	Autonomy culture, squad model, technical depth
Startups	Versatility, velocity, hands-on coding

2. Core iOS Technical Knowledge

2.1 Swift Language Mastery

Fundamentals

- Value types (`struct`, `enum`) vs. reference types (`class`)
- Optionals, optional chaining, `guard`, `if let`, `nil` coalescing
- Closures: capturing, trailing closure syntax, escaping vs. non-escaping

- Generics: type constraints, associated types, generic functions
- Protocols: protocol-oriented programming, default implementations, `where` clauses
- Error handling: `throw`, `try`, `catch`, `Result<T, E>`

Advanced Swift

- Property wrappers (`@State`, `@Published`, custom wrappers)
- Result builders (DSL construction in SwiftUI)
- `async`/`await` and structured concurrency (`Task`, `TaskGroup`, `Actor`)
- `Sendable` protocol and data-race safety
- `@MainActor`, global actors
- Macros (Swift 5.9+): `@Observable`, attached macros

Memory Management

- ARC (Automatic Reference Counting) — how it works
- Strong, weak, unowned references
- Retain cycles — detection and prevention
- Memory leaks: Instruments → Leaks / Allocations
- `[weak self]` and `[unowned self]` in closures

Skills to Gain:
- Build a library in pure Swift using generics and protocols
- Write a custom property wrapper
- Port a completion-handler-based API to `async/await`
- Identify and fix a retain cycle using Instruments

2.2 Objective-C (Awareness Level)

- Still prevalent in legacy codebases
- Interoperability with Swift: bridging headers, `@objc`, `@objcMembers`
- Categories, protocols, dynamic dispatch
- Memory management: MRC vs. ARC differences
- Key-Value Observing (KVO), Key-Value Coding (KVC)

Skills to Gain:
- Read and understand Objective-C code
- Add Swift extensions to Obj-C classes
- Know when to use `@objc` attribute

2.3 UIKit Deep Dive

View Hierarchy & Lifecycle

- `UIViewController` lifecycle: `viewDidLoad`, `viewWillAppear`, `viewDidAppear`, `viewWillDisappear`, `viewDidDisappear`
- `UIView` lifecycle: `init`, `layoutSubviews`, `drawRect`, `updateConstraints`
- Responder chain: how touch events propagate

Auto Layout

- NSLayoutConstraint, VFL, anchors
- Intrinsic content size, content hugging, compression resistance
- `UIStackView` composition
- Safe area layout guides
- Dynamic Type support

Table & Collection Views

- `UITableView` / `UICollectionView` data source and delegate patterns
- `UICollectionViewCompositionalLayout`
- `UICollectionViewDiffableDataSource` & `NSDiffableDataSourceSnapshot`
- Cell reuse and prefetching

Navigation & Routing

- `UINavigationController`, `UITabBarController`, `UISplitViewController`
- Modal presentation styles
- Deep linking and Universal Links
- Custom transitions and `UIViewControllerTransitioningDelegate`

Skills to Gain:
- Build a complex collection view with multiple sections and cell types
- Implement custom animated transitions
- Build a routing coordinator pattern

2.4 SwiftUI

Core Concepts

- Declarative UI paradigm
- `View` protocol, `body` property
- State management: `@State`, `@Binding`, `@ObservableObject`, `@EnvironmentObject`, `@Environment`
- `@Observable` macro (iOS 17+, Swift 5.9+)
- View identity and lifetime

Layout System

- `VStack`, `HStack`, `ZStack`, `LazyVStack`, `LazyHStack`
- `Grid`, `LazyGrid`
- `GeometryReader`
- `ViewThatFits` (adaptive layouts)

Advanced SwiftUI

- Custom `Layout` protocol
- `PreferenceKey` for child-to-parent communication
- `matchedGeometryEffect` for hero animations
- `Canvas` for custom drawing
- `TimelineView` for scheduled updates
- SwiftUI + UIKit interop: `UIViewRepresentable`, `UIViewControllerRepresentable`

Skills to Gain:
- Build a feature end-to-end in SwiftUI
- Migrate a UIKit screen to SwiftUI incrementally
- Implement a custom animation with `matchedGeometryEffect`

2.5 Concurrency & Threading

Grand Central Dispatch (GCD)

- Dispatch queues: serial vs. concurrent
- `DispatchQueue.main`, `DispatchQueue.global(qos:)`
- QoS classes: `.userInteractive`, `.userInitiated`, `.default`, `.utility`, `.background`
- Dispatch groups, semaphores, barriers
- Thread explosion problem

Operation Queue

- `Operation`, `OperationQueue`
- Dependencies between operations
- Cancellation

Swift Concurrency (Modern)

- `async` / `await` — sequential async code
- `Task`, `Task.detached`
- `TaskGroup` — structured parallelism
- `Actor` — data isolation
- `@MainActor` — UI updates
- `AsyncStream`, `AsyncThrowingStream`

- `Sendable` and concurrency safety
- Continuation wrapping: `withCheckedContinuation`, `withCheckedThrowingContinuation`

Skills to Gain:
- Rewrite a GCD-based network layer to Swift Concurrency
- Implement an actor to safely manage shared state
- Detect data races using Thread Sanitizer

2.6 Networking

URLSession

- `URLSession`, `URLSessionConfiguration` (default, ephemeral, background)
- `URLSessionTask` types: data, upload, download, stream, websocket
- `URLRequest`, HTTP methods, headers
- Background downloads/uploads
- Certificate pinning

Modern Networking

- `URLSession` with `async/await`
- Combine with networking
- Codable: `Encodable`, `Decodable`, custom coding keys, nested decoding
- REST API integration
- GraphQL basics (for companies using it)
- WebSockets

Network Layer Design

- Repository pattern
- Protocol-based mocking for tests
- Retry logic, exponential backoff
- Network reachability: `NWPathMonitor`

Skills to Gain:
- Build a generic, testable network layer
- Implement retry with exponential backoff
- Handle offline caching

2.7 Data Persistence

Technology	Use Case
UserDefaults	Small key-value preferences
Keychain	Secure credentials, tokens
Core Data	Complex object graphs, local database
SwiftData	Modern Core Data (iOS 17+)
SQLite / GRDB	High-performance queries
FileManager	Binary files, documents
CloudKit	iCloud sync

Core Data Deep Dive

- `NSManagedObjectContext`, `NSPersistentContainer`
- Fetch requests, predicates, sort descriptors
- Relationships and cascades
- Migration: lightweight vs. heavy migration
- Background contexts for off-main-thread operations
- `NSFetchedResultsController` for UI sync

Skills to Gain:
- Model a multi-entity Core Data schema with relationships
- Implement background fetch and merge to main context
- Perform a Core Data migration

2.8 Security

- Keychain Services API
- Biometric authentication: `LocalAuthentication` framework, Face ID / Touch ID
- Transport Layer Security: ATS (App Transport Security)
- Certificate pinning
- Jailbreak detection techniques
- Data encryption: `CryptoKit` (AES-GCM, HMAC, SHA hashing)
- OAuth 2.0 / PKCE flow for auth
- Secure coding practices: input validation, avoiding SQL injection

Skills to Gain:
- Implement biometric login with Keychain fallback

- Implement certificate pinning
- Use CryptoKit for encrypting local data

2.9 Testing

Unit Testing

- XCTest framework
- Arrange-Act-Assert pattern
- Mocking with protocols
- `@testable import`
- Test doubles: stub, mock, fake, spy

UI Testing

- `XCUIApplication`, `XCUIElement`
- Accessibility identifiers
- Page Object Model for UI tests
- Test performance

Snapshot Testing

- `swift-snapshot-testing` library
- Visual regression detection

TDD & BDD

- Test-Driven Development cycle (Red-Green-Refactor)
- Quick + Nimble for BDD-style tests
- Code coverage targets (aim for 70%+ on business logic)

Skills to Gain:
- Achieve 80%+ coverage on a service layer with unit tests
- Set up UI tests for critical user flows
- Set up snapshot tests for key UI components

---

3. iOS Architecture & Design Patterns

3.1 MVC (Model-View-Controller)

- Apple's default; leads to Massive View Controller if not disciplined
- Controller mediates Model ↔ View
- When appropriate: simple screens, prototypes

3.2 MVP (Model-View-Presenter)

- Presenter holds UI logic, View is passive
- Easier to unit test than MVC
- Boilerplate-heavy

3.3 MVVM (Model-View-ViewModel)

- ViewModel exposes state; View observes
- Pairs naturally with Combine / SwiftUI
- Works well with reactive programming
- Key: ViewModel should not import UIKit

3.4 VIPER

- View, Interactor, Presenter, Entity, Router
- Strict separation of concerns
- Common in large teams with many engineers
- Verbose but highly testable

3.5 The Composable Architecture (TCA)

- State, Action, Reducer, Store, Effect
- Unidirectional data flow
- Point-Free library
- Strong testing story
- Good for complex state machines

3.6 Clean Architecture

- Entities → Use Cases → Interface Adapters → Frameworks
- Dependency rule: inner layers know nothing about outer
- Combine with any of the above patterns

3.7 Coordinator Pattern

- Decouple navigation from ViewControllers
- `Coordinator` protocol with `start()` method
- Child coordinators for sub-flows
- Works with both UIKit and SwiftUI

3.8 Dependency Injection

- Constructor injection (preferred)
- Property injection
- DI containers: Swinject, Needle, Factory

- Avoiding service locator anti-pattern

3.9 Design Patterns (GoF Patterns in iOS Context)

Pattern	iOS Usage
Singleton	`URLSession.shared`, `NotificationCenter.default` — use sparingly
Observer	`NotificationCenter`, Combine, KVO
Delegate	`UITableViewDelegate`, custom callbacks
Factory	View/VC creation in coordinators
Builder	Complex object creation (URL builders)
Strategy	Interchangeable algorithms (sorting, networking)
Decorator	Extending behavior without subclassing
Command	Undo/redo operations

Skills to Gain:
- Refactor an MVC app to MVVM+Coordinator
- Build a feature using Clean Architecture
- Explain trade-offs of each architecture to your team

---

4. iOS Performance & Optimization

4.1 Rendering Performance

- Core Animation pipeline: app → render server → GPU
- `CALayer` vs `UIView`
- Offscreen rendering: when it happens, how to avoid
- `shouldRasterize` trade-offs
- Overdraw detection with Color Blended Layers
- Main thread rule: always update UI on main thread
- `drawRect` performance implications
- Avoiding layout recalculation on scroll

4.2 Memory Optimization

- Instruments: Allocations, Leaks, VM Tracker
- Memory warnings: `didReceiveMemoryWarning`
- Image memory: `UIImage` vs `CGImage` compression
- Lazy loading of heavy resources

- `NSCache` for memory-sensitive caching
- Avoid large objects on stack

4.3 Launch Time Optimization

- Pre-main time: dylib loading, Objective-C runtime, initializers
- Post-main time: `application(_:didFinishLaunchingWithOptions:)`
- Techniques: fewer dylibs, lazy initialization, background pre-warming
- MetricKit for launch time monitoring
- `DYLD_PRINT_STATISTICS` environment variable

4.4 Energy & Battery

- Reduce background activity
- Use `URLSession` background configurations
- Batch network requests
- Avoid polling; use push notifications
- Location: use appropriate accuracy level
- Background fetch: `BGTaskScheduler`

4.5 Network Performance

- HTTP/2 multiplexing
- Response caching: `URLCache`
- Image caching: `NSCache`, third-party (Kingfisher, SDWebImage)
- Pagination and lazy loading
- Compression (gzip)
- Protobuf vs JSON

4.6 Profiling Tools

Tool	Purpose
Instruments - Time Profiler	CPU hotspots
Instruments - Allocations	Memory allocations
Instruments - Leaks	Memory leaks
Instruments - Core Animation	Rendering performance
Instruments - Network	HTTP traffic analysis
MetricKit	Real-device production metrics
Xcode Organizer	Crash reports, energy reports
os_signpost	Custom performance points

Skills to Gain:
- Profile an app with Time Profiler and fix a CPU hotspot
- Reduce app launch time by 20%
- Detect and fix a memory leak using Instruments

---

5. Mobile DevOps & CI/CD

5.1 Build Systems

- Xcode build system fundamentals
- `xcodebuild` CLI
- Build configurations: Debug, Release, custom
- Build settings, xcconfig files
- Schemes and targets
- SPM (Swift Package Manager) vs. CocoaPods vs. Carthage

5.2 Continuous Integration

Tool	Notes
Xcode Cloud	Apple-native, tight Xcode integration
Bitrise	Mobile-first CI, rich iOS integrations
Fastlane	Automation tool for building, testing, deploying
GitHub Actions	General purpose, extensive marketplace
CircleCI	Flexible, good macOS support
Jenkins	Self-hosted, maximum control

5.3 Fastlane Essentials

- `fastlane scan` — run tests
- `fastlane gym` — build IPA
- `fastlane deliver` — upload to App Store Connect
- `fastlane match` — code signing management
- `fastlane pilot` — TestFlight management
- Lanes and custom actions

5.4 Code Signing

- Certificates: Development, Distribution
- Provisioning profiles: development, ad hoc, App Store, enterprise
- `match` for team-wide code signing
- Automatic vs. manual signing
- Apple Developer Program management

5.5 App Distribution

- TestFlight: internal vs. external testing
- App Store review process and guidelines
- Enterprise distribution
- Ad hoc distribution
- App Store Connect API

5.6 Release Engineering

- Semantic versioning (`MARKETING_VERSION`, `CURRENT_PROJECT_VERSION`)
- Feature flags: LaunchDarkly, Firebase Remote Config, custom
- Phased releases on App Store
- Rollback strategies (cannot pull from App Store; use feature flags)

- Crash monitoring: Firebase Crashlytics, Sentry, Bugsnag

Skills to Gain:
- Set up a complete Fastlane pipeline for build → test → deploy
- Configure Xcode Cloud or Bitrise for a project
- Implement feature flag infrastructure

---

6. Engineering Management Fundamentals

6.1 The Manager's Job

- **Output**: Your team's output is your output (Andy Grove)
- **Leverage**: Focus on high-leverage activities (1:1s, hiring, architecture reviews, process improvements)
- **Context**: Give context, not control; set clear goals, let engineers decide how
- **Trust**: Build trust through consistency and follow-through

6.2 Management vs. Leadership

Management	Leadership
Plans and budgets	Sets direction and vision
Organizes and staffs	Aligns people to direction
Controls and problem-solves	Motivates and inspires
Produces predictability	Produces change

Great EMs do both.

6.3 The Dual Ladder

- IC (Individual Contributor): SWE → Senior → Staff → Principal → Fellow
- Management: EM → Senior EM → Director → VP → SVP → CTO
- Understanding both paths helps with career conversations

6.4 Manager Anti-Patterns to Avoid

- **Seagull manager**: appears only to criticize, then leaves
- **Micromanager**: over-controls, kills autonomy
- **Absentee manager**: too hands-off, team drifts
- **Hero manager**: does everything themselves instead of enabling team
- **Friend over manager**: avoids hard conversations

---

7. People Management & Leadership

7.1 1:1 Meetings

Purpose:
- Relationship building
- Information gathering (team health)
- Coaching and growth
- Unblocking

Best Practices:
- Weekly for direct reports, biweekly minimum
- Engineer sets the agenda primarily
- Note-taking and follow-through essential
- Mix: short-term (tactical), long-term (growth), personal check-in

Sample 1:1 Agenda:
1. How are you doing? (personal first)
2. What's on your mind? / What's blocking you?
3. Project updates (briefly)
4. Career and growth (rotate topics)
5. Feedback exchange (two-way)

7.2 Performance Management

Setting Expectations

- Clear job levels and expectations (leveling rubrics)
- OKRs or goals set collaboratively
- Regular check-ins, not just annual reviews

Performance Reviews

- Gather peer feedback (360°)
- Self-evaluation
- Manager assessment
- Calibration with peer managers
- Written review + delivery conversation

Performance Improvement Plans (PIPs)

- Only after significant coaching
- Specific, measurable goals
- Regular check-ins
- Document everything
- Often results in separation; treat with dignity

Promotions

- Evidence-based: gather impact data proactively
- Promote for next level, not current excellence
- Know your company's promotion process and timelines
- Advocate loudly in calibrations

7.3 Hiring

Building a Strong Hiring Funnel

- Write compelling, inclusive job descriptions
- Source proactively (LinkedIn, GitHub, conferences, internal referrals)
- Partner with recruiting team
- Diverse interview panels

Interview Design

- Define what you're assessing before designing questions
- Structured interviews with consistent rubrics
- iOS technical screen: coding, architecture, debugging
- Behavioral interviews: STAR format
- Bar raiser concept (Amazon)

Candidate Experience

- Fast feedback loops
- Communicate clearly throughout
- Respect candidate's time

Onboarding

- First 30/60/90 day plan
- Buddy system
- Gradual ownership: shadow → co-pilot → solo → lead
- Early wins matter for confidence

7.4 Feedback & Coaching

SBI Model (Situation-Behavior-Impact)

- **Situation**: "In yesterday's design review..."
- **Behavior**: "...you interrupted Sarah three times..."
- **Impact**: "...which made her hesitant to share ideas."

GROW Coaching Model

- **Goal**: What do you want to achieve?

- **Reality**: What's happening now?
- **Options**: What could you do?
- **Will**: What will you do?

Radical Candor (Kim Scott)

- **Care Personally**: genuinely care about people
- **Challenge Directly**: be honest and direct
- Avoid: Ruinous Empathy, Obnoxious Aggression, Manipulative Insincerity

7.5 Team Motivation & Engagement

Intrinsic Motivators (Daniel Pink - Drive)

- **Autonomy**: control over their work
- **Mastery**: getting better at something
- **Purpose**: believing work matters

Psychological Safety (Amy Edmondson)

- Team members feel safe to take risks and speak up
- Build by: modeling vulnerability, punishing blame, rewarding learning from failure

Recognition & Appreciation

- Public recognition for impact
- Private recognition for personal growth
- Timely (not months later)
- Specific (not generic "good job")

7.6 Difficult Conversations

Framework:
1. Prepare: facts, impact, desired outcome
2. Open: state your intent clearly
3. Listen: understand their perspective
4. Explore: find root causes
5. Plan: agree on next steps
6. Follow up: check in consistently

Topics requiring difficult conversations:
- Underperformance
- Behavioral issues (communication, attitude)
- Missed promotion
- Layoff/termination
- Team conflict

---

8. Project & Program Management

8.1 Agile for Engineering Managers

Scrum Roles

- Product Owner: what to build
- Scrum Master: how the team works
- Development Team: building
- EM responsibility: often acts as facilitator and escalation path

Key Ceremonies

- Sprint planning: commit to sprint scope
- Daily standup: blockers and coordination
- Sprint review: demo to stakeholders
- Retrospective: team improvement

Kanban Alternative

- Continuous flow instead of sprints
- WIP (Work In Progress) limits
- Good for operational/support teams

8.2 Estimation & Planning

- Story points vs. time-based estimation
- T-shirt sizing for roadmap planning
- Three-point estimation: optimistic, likely, pessimistic
- Accounting for: bugs, tech debt, on-call, meetings, PTO
- Historical velocity as estimation baseline

8.3 Risk Management

- Identify risks early (technical, resource, dependency)
- Risk matrix: likelihood × impact
- Mitigation vs. contingency planning
- Communicate risks proactively to stakeholders

8.4 Technical Debt Management

- Define and categorize technical debt
- Make debt visible: track in backlog
- Negotiate dedicated debt sprints (20% of capacity as rule of thumb)
- Distinguish between reckless and prudent debt (Ward Cunningham)

8.5 Incident Management

- Severity levels (P0–P4)
- On-call rotations and schedules
- Runbooks: documented response procedures
- Post-mortems: blameless, focus on systems
- Five Whys root cause analysis
- Metrics: MTTR (Mean Time To Restore), MTTD (Mean Time To Detect)

8.6 Delivery Metrics

Metric	What It Measures
Lead Time	Idea → Production
Cycle Time	Code complete → Production
Deployment Frequency	How often you ship
Change Failure Rate	% of deployments causing failures
MTTR	Time to recover from failure

These are the DORA metrics — industry standard for measuring engineering team performance.

---

9. Cross-Functional Collaboration

9.1 Working with Product Management

- Understand the product roadmap and business context
- Push back constructively on scope creep
- Offer technical alternatives when asked to cut features
- Bring engineering constraints to roadmap discussions early
- "Build the right thing" — PMs own this, but EMs must inform it

9.2 Working with Design

- Involve engineers in design reviews early
- Raise feasibility and performance concerns before implementation
- Maintain a design system for consistency
- iOS HIG (Human Interface Guidelines) — know it well
- Accessibility standards: WCAG, `UIAccessibility`

9.3 Working with QA / Test Engineering

- Define quality standards and acceptance criteria

- Shift-left testing: testing earlier in the cycle
- Shared ownership of quality
- Flaky test management

9.4 Working with Data / Analytics

- Instrument iOS apps for tracking: Amplitude, Mixpanel, Firebase Analytics
- Define events and properties with data team
- Experiment infrastructure: A/B testing (Firebase A/B, Optimizely)
- Review A/B test results and make data-driven decisions

9.5 Working with Platform / Backend

- API versioning and iOS backward compatibility
- Contract testing: Pact
- Backend-for-frontend (BFF) pattern
- Breaking change communication and migration plans

9.6 Executive Communication

- Update format: what's happening, impact, what we need
- Speak in business outcomes, not technical details (unless asked)
- Dashboard-driven visibility: OKRs, KPIs, sprint metrics
- Never surprise executives — proactively escalate issues

---

10. System Design for Mobile

10.1 Mobile System Design Framework

Use this framework to structure any mobile system design question:

1. **Requirements clarification** (functional + non-functional)
2. **High-level architecture** (client-server interaction)
3. **Data model** (local and remote)
4. **API design** (REST endpoints, GraphQL queries)
5. **Client-side architecture** (module breakdown)
6. **Offline support** strategy
7. **Performance** (pagination, caching, lazy loading)
8. **Reliability** (error handling, retry, circuit breaker)
9. **Security** (auth, data at rest, in transit)
10. **Analytics & monitoring**

10.2 Common Mobile Design Problems

Design an Offline-First App

- Local database (Core Data / SQLite) as source of truth
- Sync engine: conflict resolution strategies (last-write-wins, CRDTs, operational transforms)
- Queue outgoing mutations for when network returns
- Optimistic UI updates

Design a Real-Time Chat App (iOS)

- WebSocket connection management
- Message ordering and deduplication
- Push notifications: APNs (Apple Push Notification service)
- Local persistence with unread counts
- Typing indicators, read receipts

Design an Image Feed (Instagram-like)

- Pagination: offset vs. cursor-based
- Image caching: `NSCache` + disk cache
- Prefetching with `UICollectionViewDataSourcePrefetching`
- Upload pipeline: compress → upload → CDN
- Optimistic rendering

Design an Offline-Capable Maps App

- Tile-based map rendering
- Tile caching strategies
- Vector vs. raster tiles
- Routing algorithm with local data

10.3 Client Architecture Design

- Modular architecture: feature modules, core modules, shared modules
- Dependency graph: avoid circular dependencies
- Binary frameworks vs. source packages
- Interface modules for decoupling
- Dynamic vs. static frameworks: trade-offs

10.4 Scaling iOS Teams

- Micro-frontend equivalent: feature flags, modular architecture
- Parallel builds: Xcode parallelization settings
- Remote build cache (Bazel, Gradle Enterprise for iOS)
- Build time optimization strategies

11. Technical Strategy & Roadmap Planning

11.1 Setting Technical Vision

- Where is the iOS platform today? (current state)
- Where should it be in 1-2 years? (future state)
- What's the gap and the path? (strategy)
- Communicate vision: ADRs (Architecture Decision Records), RFCs

11.2 Architecture Decision Records (ADRs)

Template:

```
# ADR-001: Adopt SwiftUI for new features

## Status: Accepted

## Context
Our UIKit codebase is aging. New engineers prefer SwiftUI.
SwiftUI is now stable (iOS 15+).

## Decision
All new screens will be built in SwiftUI.
UIKit remains for legacy screens until refactored.

## Consequences
- Faster UI development
- Need SwiftUI training for senior engineers
- Interop layer needed for shared navigation
```

11.3 Technology Adoption

Technology Radar Framework:
- Adopt: use in production now
- Trial: use on projects with low risk
- Assess: worth exploring in prototypes
- Hold: do not start new work with

11.4 Managing Legacy Code

- Strangler Fig pattern: replace incrementally
- Wrap before refactor: isolate legacy behind interfaces
- Boy Scout Rule: leave code better than you found it
- Measure: track % new vs. legacy code over time

11.5 Build vs. Buy Decisions

Factor	Build	Buy / Open Source
Core differentiator?	Build	—
Available quality solution?	—	Buy
Total cost of ownership	Higher	Lower (usually)
Customization needs	High	Low
Time to market	Slower	Faster

11.6 Platform Engineering for iOS

- Shared design system (fonts, colors, components)
- Common networking and auth modules
- Analytics SDK abstraction
- Error handling and logging standards
- Feature flag infrastructure

---

12. Data-Driven Engineering

12.1 Key iOS Metrics

User-Facing Metrics

- Crash-free sessions rate (target: >99.5%)
- App launch time (cold/warm/hot launch)
- ANR / Hang rate (watchdog terminations, main thread hangs)
- Screen load time (P50, P90, P99)
- Network error rate

Business Metrics

- DAU/MAU (Daily/Monthly Active Users)
- Retention (D1, D7, D30)
- Session length and depth
- Conversion rates (funnel analysis)
- Feature adoption rate

Team Metrics

- Cycle time and lead time
- Deployment frequency

- Bug escape rate
- Test coverage
- Build time

12.2 Instrumentation

- Event tracking: user actions, screen views, errors
- Custom MetricKit metrics
- Logging: `os_log`, Unified Logging System
- Distributed tracing: correlate iOS events with backend traces

12.3 A/B Testing

- Hypothesis: "We believe X will improve Y by Z"
- Experiment setup: control vs. treatment group
- Statistical significance: p-value < 0.05, power ≥ 0.8
- Guard rails: monitor for negative side effects
- Ship or kill: data-driven decisions

12.4 Dashboards & Visibility

- Build dashboards for: releases, crashes, performance, product metrics
- Share with team and stakeholders
- Alert on regressions (automated)
- Weekly team metrics review

---

13. Interview Question Bank

13.1 Technical iOS Questions

Architecture:
- "How would you choose between MVVM and VIPER for a new iOS project?"
- "Explain the coordinator pattern. How does it solve the massive view controller problem?"
- "How do you handle dependency injection at scale in an iOS app?"
- "Describe how you would migrate a UIKit app to SwiftUI incrementally."

Concurrency:
- "Explain the difference between GCD and Swift concurrency. When would you choose one over the other?"
- "What are actors in Swift concurrency and what problem do they solve?"
- "How do you prevent main thread blocking in a UIKit app?"
- "What is a retain cycle and how does [weak self] solve it in a closure?"

Performance:
- "How would you investigate and fix an app that users say 'feels slow'?"

- "Explain the Core Animation rendering pipeline."
- "How do you optimize table view scrolling performance?"
- "What causes app launch time to increase and how do you fix it?"

Networking:
- "Design a networking layer for a production iOS app."
- "How would you implement offline support in an iOS app?"
- "Explain certificate pinning and when you'd use it."

13.2 Engineering Management Behavioral Questions

Leadership & People:
- "Tell me about a time you turned around an underperforming engineer."
- "How do you handle a situation where your best engineer wants to leave?"
- "Describe how you've built a high-performing team from scratch."
- "Tell me about a time you had to make an unpopular decision with your team."
- "How do you balance technical work with management responsibilities?"

Delivery & Execution:
- "Tell me about a project that was severely behind schedule. How did you handle it?"
- "How do you manage technical debt while maintaining velocity?"
- "Describe a time you had to negotiate scope with a product manager."
- "How do you ensure quality without slowing down your team?"

Conflict & Communication:
- "Tell me about a conflict between two senior engineers on your team. How did you resolve it?"
- "How do you handle disagreement with your own manager?"
- "Describe a time you had to deliver difficult feedback to an engineer."
- "How do you communicate a project delay to stakeholders?"

Strategy & Vision:
- "How do you balance short-term delivery with long-term technical health?"
- "Tell me about a major technical decision you drove for your iOS platform."
- "How do you stay current with iOS platform changes while managing a team?"
- "Describe how you built the technical roadmap for your iOS team."

Hiring:
- "How do you assess iOS engineering talent in an interview?"
- "Tell me about a great hire you made and what made them great."
- "How do you build a diverse and inclusive engineering team?"
- "What makes a great onboarding experience for a new iOS engineer?"

13.3 System Design Interview Questions

- "Design the iOS client architecture for a social media feed app."
- "Design an offline-first to-do app for iOS."
- "How would you design the push notification system for a chat app?"
- "Design the iOS architecture for a real-time collaborative document editor."

- "How would you design the iOS SDK for a third-party analytics provider?"

13.4 STAR Answer Templates

STAR Format: Situation → Task → Action → Result

Example: Managing underperformance

> ***Situation**: One of my senior iOS engineers was consistently missing deadlines and producing code with high bug counts. Team morale was starting to dip.*
> ***Task**: I needed to address the performance issue while preserving team trust and being fair to the engineer.*
> ***Action**: I had a candid 1:1 conversation, gathered specific examples with data, collaborated on a PIP with clear 30-day goals, provided weekly check-ins with coaching, and looped in HR. I also helped the engineer understand the why — they were struggling with a personal situation I wasn't aware of.*
> ***Result**: The engineer improved significantly over 45 days, completed the PIP successfully, and has since led two major features. Team morale improved as they saw the issue being addressed directly.*

---

14. 30-60-90 Day Plan

First 30 Days: Listen & Learn

Goal: Build trust and understand the current state

Week 1–2 (Orientation):
- Meet with every direct report (1:1, discovery conversation)
- Meet with key stakeholders: PM, Design, QA leads, senior engineers
- Review existing iOS codebase, architecture docs, ADRs
- Understand CI/CD pipeline end-to-end
- Review crash reports, performance dashboards, App Store reviews

Week 3–4 (Diagnosis):
- Shadow team in standups, sprint planning, design reviews
- Review last 3 sprint retrospectives
- Understand current technical debt inventory
- Identify top 3 team pain points
- Review open bugs and P0/P1 incident history

Deliverable by Day 30: Written "State of the iOS Team" doc with observations and early hypotheses

---

Days 31–60: Build Relationships & Early Wins

Goal: Establish credibility and begin shaping

- Establish regular 1:1 cadence and fill in team member growth plans
- Drive one quick win (e.g., fix broken CI pipeline, automate a manual process)

- Begin structured code reviews to understand code quality
- Participate actively in architecture discussions
- Set up metrics dashboards for team visibility
- Identify one area for process improvement and pilot it

Deliverable by Day 60: Updated team goals aligned with product roadmap; first engineering OKRs draft

---

Days 61–90: Set Direction & Deliver

Goal: Lead from the front with a clear vision

- Define iOS technical roadmap for next 2 quarters
- Introduce one architectural improvement initiative
- Run team health survey and share results with team
- Launch structured interview process improvements
- Deliver first complete sprint cycle as EM
- Present iOS platform strategy to engineering leadership

Deliverable by Day 90: Presented iOS Technical Strategy deck; team OKRs finalized; clear roadmap communicated

---

15. Learning Resources & Skill-Building Plan

Phase 1: iOS Technical Mastery (Months 1–3)

Topic	Action
Swift advanced features	Build a real app using async/await, actors, Combine
SwiftUI	Complete a project from scratch in SwiftUI
Core Data / SwiftData	Build a local-first app with full CRUD and migration
Testing	Achieve 80%+ coverage on a new module
Performance	Profile a real app with Instruments; fix 2 issues
Architecture	Refactor an MVC app to MVVM+Coordinator
CI/CD	Set up a complete Fastlane pipeline from scratch

Phase 2: Management Fundamentals (Months 2–4)

Topic	Action
1:1 structure	Run structured 1:1s and document learnings
Giving feedback	Practice SBI model in low-stakes situations
Conflict resolution	Role-play difficult conversations
Hiring	Conduct 10+ technical interviews with structured rubrics
Performance reviews	Write sample performance reviews for hypothetical engineers
Goal setting	Write OKRs for a hypothetical iOS team

Phase 3: Strategy & Leadership (Months 3–6)

Topic	Action
Technical vision	Write a 12-month iOS platform strategy doc
Architecture decisions	Write 5 ADRs for real or hypothetical decisions
System design	Practice 10 mobile system design problems
Executive communication	Present a mock iOS status update to senior stakeholders
Data-driven decisions	Analyze A/B test results and write a decision memo
Cross-functional	Simulate PM/EM collaboration on a feature

Phase 4: Interview Preparation (Months 4–6)

Activity	Frequency
STAR answer preparation	Write 20 STAR stories covering all key behavioral themes
Technical coding practice	2–3 LeetCode/Swift Coding problems per week
System design practice	2 mobile system designs per week with peer review
Mock interviews	5+ mock EM interviews with engineers or coaches
Role-specific research	Research target company's iOS architecture and culture
Applied networking	Connect with EMs at target companies on LinkedIn

16. Books, Courses & Communities

Essential Books for iOS Engineering Managers

Management & Leadership

Book	Author	Key Value
The Manager's Path	Camille Fournier	Best intro to engineering management, EM-specific
High Output Management	Andy Grove	Management fundamentals from Intel CEO
An Elegant Puzzle	Will Larson	Engineering management at scale
Radical Candor	Kim Scott	Feedback and leadership framework
Accelerate	Nicole Forsgren	Data-driven engineering, DORA metrics
The Five Dysfunctions of a Team	Patrick Lencioni	Team dynamics and trust
Drive	Daniel Pink	Motivation and autonomy
Crucial Conversations	Patterson et al.	Handling difficult conversations
First, Break All the Rules	Marcus Buckingham	Great manager behaviors from Gallup research
No Rules Rules	Reed Hastings	Netflix culture and high performance

iOS & Technical

Book	Author	Key Value
Swift in Depth	Tjeerd in 't Veen	Advanced Swift patterns
iOS Unit Testing by Example	Jon Reid	Test-driven iOS development
Advanced iOS App Architecture	Rene Cacheaux	Architecture patterns in depth
Swift Design Patterns	Paul Hudson	Design patterns in Swift
Designing Data-Intensive Applications	Martin Kleppmann	Backend systems knowledge for EMs
A Philosophy of Software Design	John Ousterhout	Code complexity and design
Clean Architecture	Robert C. Martin	Architecture principles

Online Courses

Platform	Course	Purpose
Apple Developer	WWDC sessions	Latest iOS APIs and best practices
Point-Free	pointfree.co	Functional Swift, TCA, Composable Architecture
Ray Wenderlich / Kodeco	iOS courses	Comprehensive iOS topics
Stanford CS193p	SwiftUI App Development	Free, university-level SwiftUI
Pluralsight	iOS architecture courses	Architecture deep dives
Coursera	Engineering Management specializations	Management frameworks
LinkedIn Learning	People management courses	Soft skills development

Platforms & Practice

Platform	Use
GitHub	Build portfolio projects, contribute to open source
LeetCode (Swift)	Coding interview practice
Excalidraw / Miro	System design whiteboarding practice
Newsletter: iOS Dev Weekly	Stay current with iOS ecosystem
Newsletter: Lenny's Newsletter	Product and engineering leadership
Newsletter: The Pragmatic Engineer	Engineering management insights (Gergely Orosz)

Communities

Community	Value
iOS Dev Happy Hour (Slack)	iOS developer community
Swift Forums (forums.swift.org)	Swift language evolution discussions
Indie Dev Monday	Indie iOS developers
iOSLeadEssentials.com	iOS architecture and career community
Engineering Managers Slack (Rands)	Rands Leadership Slack — large EM community
LinkedIn groups	Engineering Management, iOS Development groups
Local iOS Meetups / NSConference	In-person networking
WWDC (Apple Developer Conference)	Annual Apple platform updates

Podcasts

Podcast	Focus
Swift by Sundell	iOS development best practices
iOS Dev Discussions	iOS career and technical topics
Stacktrace	iOS and Apple ecosystem
Developing Leadership	Engineering leadership podcast
The Engineering Manager	EM-focused podcast
Soft Skills Engineering	Non-technical career advice for engineers
Lenny's Podcast	Product and growth (EM-relevant)

Quick Reference: Interview Readiness Checklist

Technical iOS (Self-Assessment)

- ☐ Can explain ARC, retain cycles, and weak/unowned confidently
- ☐ Have implemented async/await and actors in a real project
- ☐ Can design a production-grade networking layer from scratch
- ☐ Know MVVM, VIPER, TCA trade-offs and can explain them clearly
- ☐ Have diagnosed and fixed a memory leak using Instruments

- ☐ Have optimized app launch time
- ☐ Understand Core Data migrations
- ☐ Can set up a CI/CD pipeline with Fastlane
- ☐ Have built features in both UIKit and SwiftUI
- ☐ Understand certificate pinning and Keychain usage

Management (Self-Assessment)

- ☐ Have managed at least 4 direct reports (preferred)
- ☐ Have conducted 10+ technical interviews
- ☐ Have delivered performance reviews and difficult feedback
- ☐ Have driven a project from planning to production
- ☐ Have managed technical debt negotiation with product
- ☐ Have experienced or led incident response (on-call)
- ☐ Have hired and onboarded engineers
- ☐ Can run a blameless post-mortem
- ☐ Have set OKRs or team goals
- ☐ Have influenced without authority across teams

Behavioral STAR Stories Prepared

- ☐ Technical decision I drove and its impact
- ☐ Conflict I resolved between engineers
- ☐ Underperforming engineer I helped improve
- ☐ Project that was at risk and how I saved it
- ☐ Unpopular decision I made and how I handled pushback
- ☐ Great hire I made
- ☐ Technical debt strategy I implemented
- ☐ Cross-functional win I delivered
- ☐ Promotion I advocated for (or denied, with reasoning)
- ☐ Culture change I drove on the team

Key Frameworks Summary Card

Framework	Use
STAR	Behavioral interview answers
SBI	Giving specific feedback
GROW	Coaching conversations
DORA Metrics	Measuring team performance
RACI	Defining ownership and accountability
RFC / ADR	Technical decision documentation
OKR	Goal setting and alignment
Radical Candor	Feedback culture
Technology Radar	Technology adoption decisions
Strangler Fig	Legacy code migration

This document is a living reference. Revisit and update it as you grow in your Engineering Manager journey.

iOS Technical Depth Guide for Engineering Managers

> *This document provides deep-dive technical content that an EM - iOS must know or be able to discuss confidently in interviews and with their team.*

Swift Concurrency: Complete Mental Model

The Problem Swift Concurrency Solves

Before Swift 5.5, async code used:
1. Completion handlers → callback hell, error prone
2. Delegate pattern → scattered code
3. Combine → steep learning curve, FRP paradigm

```
// OLD: Completion handler pyramid of doom
func loadUser(id: String, completion: @escaping (Result<User, Error>) → Void) {
    fetchUser(id: id) { result in
        switch result {
        case .success(let user):
            fetchAvatar(url: user.avatarURL) { avatarResult in
                switch avatarResult {
                case .success(let image):
                    fetchPosts(userId: user.id) { postsResult in
                        // Deeply nested, hard to read/maintain
                    }
                case .failure(let error):
                    completion(.failure(error))
                }
            }
        case .failure(let error):
            completion(.failure(error))
        }
    }
}

// NEW: Swift Concurrency - reads like synchronous code
func loadUser(id: String) async throws → (User, UIImage, [Post]) {
    let user = try await fetchUser(id: id)
    async let avatar = fetchAvatar(url: user.avatarURL)
    async let posts = fetchPosts(userId: user.id)
    return try await (user, avatar, posts) // parallel execution
}
```

Task Hierarchy & Structured Concurrency

```
Task (root)
├── child Task 1
│   ├── grandchild Task A
│   └── grandchild Task B
└── child Task 2
```

Cancellation propagates down: if parent is cancelled, all children are cancelled automatically.

Error propagation goes up: if a child throws, the parent's scope handles it.

Actor Isolation - The Real Model

```
// WITHOUT actor - data race possible
class UnsafeCounter {
    var count = 0
    func increment() { count += 1 } // NOT thread-safe
}

// WITH actor - compiler-enforced isolation
actor SafeCounter {
    var count = 0
    func increment() { count += 1 } // Safe - only one caller at a time
    func getCount() → Int { return count }
}

// Calling from outside requires await
let counter = SafeCounter()
await counter.increment()
let value = await counter.getCount()
```

MainActor — The UI Thread Contract

```
// Marking an entire class as @MainActor
@MainActor
class ViewModel: ObservableObject {
    @Published var items: [Item] = []

    // All methods run on main thread
    func updateUI(with items: [Item]) {
        self.items = items // Safe - we're on MainActor
    }

    // Use nonisolated for non-UI work
    nonisolated func computeHeavyWork() async → [Item] {
        // Runs on a background thread
        return await Task.detached(priority: .background) {
            // heavy computation
            return []
        }.value
    }
}
```

UIKit vs SwiftUI: When to Use Each

Decision Matrix

Scenario	UIKit	SwiftUI	Notes
Legacy codebase	✅	⚠️	Interop available but complex
iOS 13+ minimum deployment	⚠️	✅	SwiftUI stable from iOS 14/15
Complex custom animations	✅	⚠️	UIKit more control
Simple declarative UI	⚠️	✅	SwiftUI shines here
Collection with complex layout	✅	⚠️	UICollectionView more flexible
Forms and settings UI	⚠️	✅	SwiftUI Form is excellent
Accessibility	Both	Both	Both support well
Watchdog/Memory sensitivity	Both	Both	SwiftUI can be heavier

Interoperability Pattern

```
// Using a UIKit view inside SwiftUI
struct MapView: UIViewRepresentable {
    @Binding var region: MKCoordinateRegion

    func makeUIView(context: Context) -> MKMapView {
        let mapView = MKMapView()
        mapView.delegate = context.coordinator
        return mapView
    }

    func updateUIView(_ uiView: MKMapView, context: Context) {
        uiView.setRegion(region, animated: true)
    }

    func makeCoordinator() -> Coordinator {
        Coordinator(self)
    }

    class Coordinator: NSObject, MKMapViewDelegate {
        var parent: MapView
        init(_ parent: MapView) { self.parent = parent }
    }
}

// Using a SwiftUI view inside UIKit
class HostingViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        let swiftUIView = MySwiftUIView()
        let hostingController = UIHostingController(rootView: swiftUIView)
        addChild(hostingController)
        view.addSubview(hostingController.view)
        hostingController.didMove(toParent: self)
        // Set up constraints...
    }
}
```

Core Data: Production Patterns

Proper Stack Setup

```
class CoreDataStack {
    static let shared = CoreDataStack()

    lazy var persistentContainer: NSPersistentContainer = {
        let container = NSPersistentContainer(name: "MyModel")

        // WAL mode for better concurrency
        let description = container.persistentStoreDescriptions.first!
        description.shouldAddStoreAsynchronously = true
        description.setOption(true as NSNumber,
                              forKey: NSPersistentHistoryTrackingKey)

        container.loadPersistentStores { _, error in
            if let error = error {
                fatalError("Core Data failed: \(error)")
            }
        }
        container.viewContext.automaticallyMergesChangesFromParent = true
        return container
    }()

    // Background context for writes
    func newBackgroundContext() → NSManagedObjectContext {
        let context = persistentContainer.newBackgroundContext()
        context.mergePolicy = NSMergeByPropertyObjectTrumpMergePolicy
        return context
    }

    // Perform writes in background
    func performBackgroundTask(_ block: @escaping (NSManagedObjectContext) → Void) {
        persistentContainer.performBackgroundTask(block)
    }
}
```

Fetch with Predicate — Performance Patterns

```
// Fetch request with index-optimized predicate
func fetchRecentOrders(for userId: String, limit: Int = 50) → [Order] {
    let request = Order.fetchRequest()

    // Use indexed attributes in predicates
    request.predicate = NSPredicate(format: "userId == %@ AND status ≠ %@",
                                    userId, "cancelled")
    request.sortDescriptors = [NSSortDescriptor(key: "createdAt", ascending: false)]
    request.fetchLimit = limit

    // Only fetch what you need
    request.propertiesToFetch = ["orderId", "total", "createdAt", "status"]
    request.resultType = .dictionaryResultType

    do {
        return try viewContext.fetch(request)
    } catch {
        logger.error("Fetch failed: \(error)")
        return []
    }
}
```

Networking Layer Architecture

Production-Grade Network Layer

```
// Protocol for testability
protocol NetworkClient {
    func request<T: Decodable>(_ endpoint: Endpoint) async throws -> T
}

// Endpoint definition
struct Endpoint {
    let path: String
    let method: HTTPMethod
    let headers: [String: String]
    let body: Encodable?
    let queryItems: [URLQueryItem]?
}

// Real implementation
final class URLSessionNetworkClient: NetworkClient {
    private let session: URLSession
    private let baseURL: URL
    private let tokenProvider: TokenProvider

    func request<T: Decodable>(_ endpoint: Endpoint) async throws -> T {
        var request = try buildRequest(from: endpoint)
        request.addValue("Bearer \(try await tokenProvider.token())",
                         forHTTPHeaderField: "Authorization")

        let (data, response) = try await session.data(for: request)

        guard let httpResponse = response as? HTTPURLResponse else {
            throw NetworkError.invalidResponse
        }

        switch httpResponse.statusCode {
        case 200...299:
            return try JSONDecoder().decode(T.self, from: data)
        case 401:
            throw NetworkError.unauthorized
        case 429:
            throw NetworkError.rateLimited
        default:
            throw NetworkError.serverError(httpResponse.statusCode)
        }
    }
}

// Retry with exponential backoff
extension NetworkClient {
    func requestWithRetry<T: Decodable>(
        _ endpoint: Endpoint,
        maxAttempts: Int = 3
    ) async throws -> T {
        var lastError: Error?
        for attempt in 0..<maxAttempts {
            do {
                return try await request(endpoint)
            } catch NetworkError.serverError(let code) where code ≥ 500 {
                lastError = NetworkError.serverError(code)
                let delay = Double(pow(2.0, Double(attempt))) // 1s, 2s, 4s
                try await Task.sleep(nanoseconds: UInt64(delay * 1_000_000_000))
```

```
            } catch {
                throw error // Don't retry client errors
            }
        }
        throw lastError!
    }
}
```

Testing Strategy for iOS Teams

Testing Pyramid for iOS

```
        /\
       /  \
      / E2E\       ← XCUITest (few, high value)
     /------\
    /  Integ  \    ← Integration tests (some)
   /------------\
  /  Unit Tests  \ ← Fast, many, no dependencies
 /----------------\
```

Unit Testing with Mocks

```
// Protocol for testability
protocol UserRepository {
    func fetchUser(id: String) async throws → User
    func saveUser(_ user: User) async throws
}

// Production implementation
final class RemoteUserRepository: UserRepository {
    private let client: NetworkClient

    func fetchUser(id: String) async throws → User {
        try await client.request(.getUser(id: id))
    }

    func saveUser(_ user: User) async throws {
        try await client.request(.updateUser(user))
    }
}

// Mock for tests
final class MockUserRepository: UserRepository {
    var fetchedUsers: [String: User] = [:]
    var savedUsers: [User] = []
    var shouldThrowError: Error?

    func fetchUser(id: String) async throws → User {
        if let error = shouldThrowError { throw error }
        return fetchedUsers[id] ?? User.mock()
    }

    func saveUser(_ user: User) async throws {
        if let error = shouldThrowError { throw error }
        savedUsers.append(user)
    }
}

// Test
final class ProfileViewModelTests: XCTestCase {
    var sut: ProfileViewModel!
    var repository: MockUserRepository!

    override func setUp() {
        repository = MockUserRepository()
        sut = ProfileViewModel(repository: repository)
    }

    func testLoadProfile_success_updatesState() async throws {
        // Arrange
        let expectedUser = User(id: "123", name: "Alice", email: "alice@example.com")
        repository.fetchedUsers["123"] = expectedUser

        // Act
        await sut.loadProfile(id: "123")

        // Assert
        XCTAssertEqual(sut.user, expectedUser)
        XCTAssertFalse(sut.isLoading)
        XCTAssertNil(sut.error)
```

```
    }

    func testLoadProfile_failure_setsError() async throws {
        // Arrange
        repository.shouldThrowError = NetworkError.unauthorized

        // Act
        await sut.loadProfile(id: "123")

        // Assert
        XCTAssertNil(sut.user)
        XCTAssertFalse(sut.isLoading)
        XCTAssertNotNil(sut.error)
    }
}
```

Performance Profiling Workflow

Step-by-Step: Debug a Slow Scroll

1. **Reproduce on device** (not simulator — GPU/CPU behavior differs)
2. **Open Instruments** → Core Animation template
3. **Look for**: frames below 60fps (16.67ms), red bars in Frame Rate track
4. **Common culprits**:
    - Offscreen rendering (blending, shadows, rounded corners with `masksToBounds`)
    - Fat `cellForRowAt` — heavy work on main thread
    - Unoptimized images — wrong size/format
    - Auto Layout constraint solving overhead
5. **Fix: Offscreen rendering**

```
// BEFORE: Causes offscreen rendering
cell.avatarView.layer.cornerRadius = 25
cell.avatarView.layer.masksToBounds = true  // offscreen render trigger
cell.avatarView.layer.shadowOffset = CGSize(width: 2, height: 2)
cell.avatarView.layer.shadowOpacity = 0.3

// AFTER: Use CALayer shadow path (no offscreen render) + pre-clipped image
cell.avatarView.layer.cornerRadius = 25
cell.avatarView.layer.masksToBounds = true
// Move shadow to a container view WITHOUT masksToBounds
cell.shadowContainer.layer.shadowPath = UIBezierPath(roundedRect: ...).cgPath
cell.shadowContainer.layer.shadowOpacity = 0.3
```

1. **Fix: Heavy cellForRowAt**

```
// BEFORE
func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) → UITableViewCell {
    let cell = ...
    // Heavy work on main thread
    let processedImage = ImageProcessor.applyFilter(to: rawImage) // BAD
    let attributedString = buildAttributedString(from: longText)  // BAD
    return cell
}

// AFTER: Pre-compute and cache
func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) → UITableViewCell {
    let cell = ...
    let viewModel = viewModels[indexPath.row] // pre-computed
    cell.configure(with: viewModel)           // just assign, no computation
    return cell
}
```

Modular iOS Architecture at Scale

Module Types

```
App Target
├── Feature Modules (vertical slices)
│   ├── FeatureHome
│   ├── FeatureProfile
│   ├── FeatureCheckout
│   └── FeatureSearch
├── Core Modules (shared infrastructure)
│   ├── Networking
│   ├── Analytics
│   ├── Authentication
│   └── CoreData
└── UI Module
    ├── DesignSystem (colors, fonts, tokens)
    └── SharedComponents (buttons, cards, etc.)
```

Why Modularize?

1. **Build time**: only recompile changed modules
2. **Team ownership**: teams own specific modules
3. **Testability**: modules are independently testable
4. **Reusability**: share across app and extensions (widget, watchOS)
5. **Enforces boundaries**: prevents unauthorized dependencies

Dependency Direction Rules

```
Feature → Core, UI        ✅
Core → Core               ✅ (if no cycles)
Feature → Feature         ❌ (use shared abstractions)
Core → Feature            ❌ (inner layer cannot know outer)
UI → Feature              ❌
```

Common iOS EM Technical Interview: What They Really Test

When They Ask Technical Questions

Companies aren't testing if you can still write production code daily. They are testing:

1. **Credibility**: Can you earn engineers' respect?
2. **Judgment**: Can you make the right technical call?
3. **Communication**: Can you explain complex things simply?
4. **Growth**: Are you current with the platform?
5. **Trade-offs**: Do you understand cost vs. benefit?

How to Answer Technical Architecture Questions

Structure:
1. Restate the problem and clarify constraints
2. Identify key requirements (scale, latency, offline, etc.)
3. Propose a high-level approach
4. Deep-dive into the interesting/hard parts
5. Discuss alternatives and trade-offs
6. Mention what you'd monitor/measure

Example: "How would you architect offline support for a mobile banking app?"

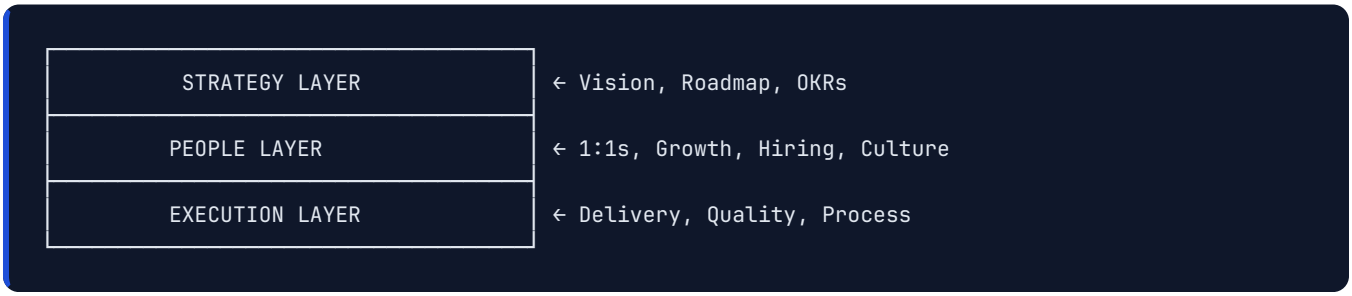
> *"Great question. Let me clarify scope first — are we talking read-only offline (view last synced balance) or read-write offline (initiate transactions offline)?*
> *For read-write offline in a banking context, I'd treat the local database as the source of truth. I'd use Core Data with CloudKit or a custom sync engine. For outgoing mutations — like payment initiation — I'd queue them in a local `PendingOperations` table with idempotency keys. When connectivity restores, the sync engine replays them.*
> *Conflict resolution in banking is tricky. I'd use an optimistic UI with server-side validation as the authority. Failed conflicts surface as non-dismissible error banners.*
> *I'd instrument this with `NWPathMonitor` for network state and `os_signpost` for sync performance. I'd track sync success rate, queue depth, and conflict frequency as operational metrics."*

Engineering Manager Playbook: People, Process & Leadership

> *Practical frameworks, templates, and tactics for day-to-day engineering management.*

The EM Operating System

Think of your role as running a system with three layers:



A great EM keeps all three layers healthy simultaneously. When one breaks down, the whole system degrades.

1:1 Mastery

The 1:1 Is Not a Status Update

Status is asynchronous (Slack, Jira, email). The 1:1 is for:
- **Trust building**: the relationship is the foundation of everything
- **Coaching**: help them grow, unblock, think through problems
- **Feedback**: both directions
- **Early signals**: catch burnout, conflict, and confusion early
- **Career**: understand their aspirations and help them get there

1:1 Agenda Template

```
## 1:1 - [Name] - [Date]

### Their Agenda (engineer fills before meeting)
-

### Manager Agenda
-

### Standing Items (rotate weekly)
Week 1: Career goals and growth
Week 2: Team/project health check
Week 3: Feedback exchange
Week 4: Longer-term aspirations

### Action Items (carry forward)
- [ ]
- [ ]

### Notes
```

Discovery Questions for New Direct Reports

Use these in the first 2-4 weeks:
- "What's the best thing about working here that you'd never want to change?"
- "What's the one thing that drives you most crazy about how the team works?"
- "When have you felt most proud of your work in this team?"
- "What does your ideal working relationship with a manager look like?"
- "What are you hoping to get better at in the next year?"
- "Is there anything about the team dynamics I should know about?"
- "What would make you consider leaving this team?"
- "If you were the EM, what would you do differently?"

Warning Signs in 1:1s

Signal	Possible Meaning
Engineer cancels repeatedly	Disengagement, or too busy (overloaded)
Only short, surface answers	Low trust, defensive
Complaining without proposing	Frustration building
"Everything is fine" always	Not psychologically safe enough to share
Energy drop from previous meetings	Something happened — ask
Talking about other jobs/companies	Retention risk — explore motivators

Career Development Framework

Career Conversation Structure (3 Conversations)

Based on Russ Laraway's "Career Conversations" model:

Conversation 1: Life Story
- "Tell me about your life from childhood to now — the significant choices and experiences."
- Goal: understand their values, what drives them, how they make decisions
- Not about work — about the person

Conversation 2: Dreams
- "What's your ultimate dream — even if it seems impossible?"
- Explore multiple options: "What else? What else?"
- Goal: understand where they genuinely want to go
- Not yet about what they should do — about inspiration

Conversation 3: 18-Month Plan
- Connect life story + dreams to immediate priorities
- What skills do they need to develop?
- What roles/projects will accelerate growth?
- What does success look like in 18 months?

Growth Plan Template

```
## Growth Plan: [Engineer Name]
## Period: Q3 2024 - Q4 2024

### Current Level: iOS Engineer II
### Target: iOS Engineer III (promotion-ready by end of period)

### Promotion Criteria (company rubric)
- Technical: leads feature design, mentors junior engineers, drives architecture decisions
- Execution: delivers complex projects with minimal guidance
- Impact: work influences beyond immediate team

### Gap Analysis
| Area | Current State | Target State |
|---|---|---|
| System Design | Designs components | Designs features/systems |
| Mentorship | None yet | Actively mentoring 1 junior |
| Initiative | Waits for direction | Identifies and drives improvements |
| Scope | Feature-level | Cross-team collaboration |

### Development Actions (This Quarter)
1. **Lead the offline sync feature** — own design, execution, and stakeholder communication
2. **Mentor [Junior Engineer]** — weekly pair sessions, code reviews
3. **Present at team tech talk** — pick iOS architecture topic

### Milestones
- Month 1: Feature design doc reviewed and approved
- Month 2: Implementation complete, junior mentorship in progress
- Month 3: Feature shipped, tech talk done, promotion doc drafted

### EM Support
- Introduce to senior engineers in platform team for collaboration
- Include in architecture review sessions
- Provide stretch opportunities in Q4 planning
```

Promotion Advocacy Script

Use this framework when advocating in calibration sessions:

> *"[Name] is ready for [Level N+1]. Here's the evidence:*
> ***Technical impact**: Led the redesign of our offline sync module — reduced data conflicts by 40%, shipped to 100% of users. This was a system-level decision affecting 4 other teams.*
> ***Leadership**: Has been the de facto tech lead for the payments feature for 2 quarters. Brings engineers up-front in design, runs code reviews that catch architectural issues.*
> ***Scope**: Drove adoption of our new testing standards across the mobile team — not just their own work.*
> *This isn't about how long they've been here — they are clearly operating at the next level and have been for 6 months. I'm proposing a promotion."*

Hiring: End-to-End Process

iOS Engineer Interview Loop

Round 1: Recruiter Screen (recruiter owns)
- Motivation, logistics, compensation alignment

Round 2: Technical Phone Screen (senior engineer)
- 45 min Swift coding: medium-complexity problem
- Focus: code quality, communication, Swift idioms
- Not LeetCode puzzles — real-world iOS problem

Round 3: Technical Loop (3-4 sessions)
- Session A: iOS Architecture (design a feature from scratch)
- Session B: Coding / debugging session
- Session C: System Design (mobile system design)
- Session D (optional): Cross-functional / product collaboration

Round 4: Management Interview (EM owns)
- Culture fit, values alignment
- Team collaboration, conflict resolution
- Career motivations
- Past project deep-dive

Round 5: Offer & Close (recruiter + EM)

iOS Architecture Interview: Rubric

Question: "Design the iOS app architecture for a news feed with offline support."

Dimension	Weak	Strong	Exceptional
Requirements	Jumps in	Asks some questions	Clarifies functional + non-functional + constraints
Architecture	Names a pattern	Explains pattern + why	Discusses trade-offs of 2-3 options with justification
Offline	"Use CoreData"	Explains sync strategy	Discusses conflict resolution, queue, idempotency
Performance	Not mentioned	Mentions pagination	Discusses image caching, prefetching, lazy loading
Testing	Not mentioned	Mentions unit tests	Explains how architecture enables testing
Communication	Unclear	Clear	Draws diagrams proactively, confirms understanding

Hiring Debrief Protocol

After each interview, each interviewer independently submits:
- Hire / Lean Hire / Lean No Hire / No Hire recommendation
- Evidence: specific quotes, observations, code quality notes
- Key signals for and against

Debrief meeting (within 24 hours):
1. Each interviewer shares recommendation without hearing others first (to reduce anchoring)
2. Discuss discrepancies
3. Identify must-have vs. nice-to-have signal gaps
4. Align on decision
5. EM makes final call and communicates to recruiter

Offer Negotiation Tips for EMs

- Know your compensation band before the debrief
- Understand the candidate's competing offers if possible
- Differentiate your offer: project impact, team quality, growth opportunities
- Don't create false urgency — builds resentment
- Be willing to walk away if compensation expectations don't align

---

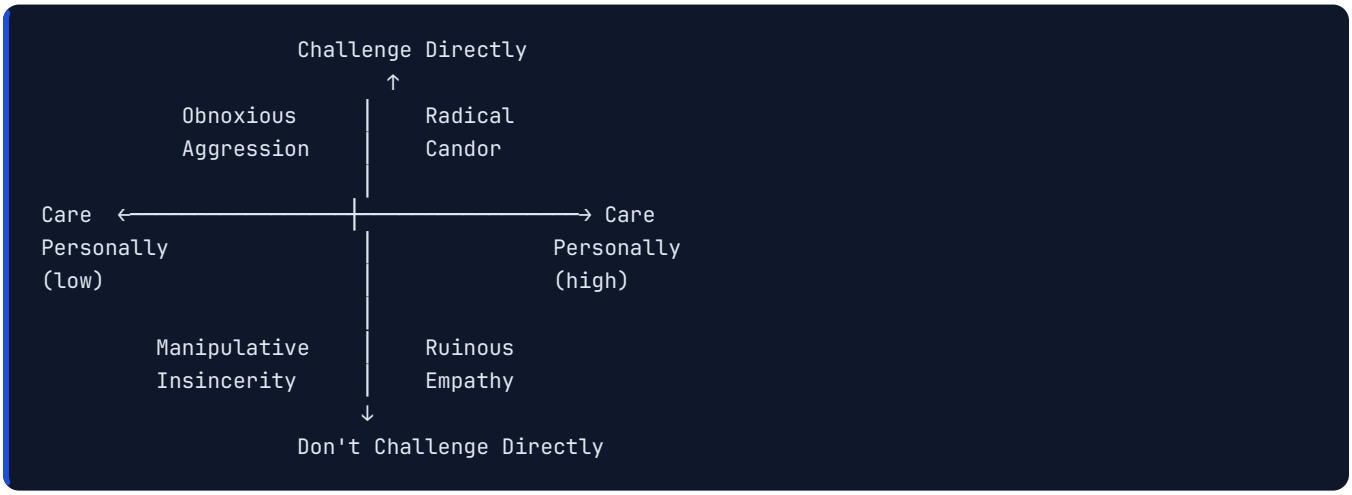
Delivering Difficult Feedback

SBI Framework in Practice

Situation-Behavior-Impact:

Bad Feedback	Good Feedback (SBI)
"You're too aggressive in meetings"	"In yesterday's architecture review (S), you interrupted Maria three times before she finished her point (B). She stopped contributing after that, and we may have missed her perspective (I)."
"Your code quality is bad"	"In last week's PR (S), there were no unit tests for the payment flow (B). This led to a P1 bug in production that took 4 hours to debug (I)."
"You need to communicate better"	"During the sprint (S), you didn't update the Jira tickets or Slack the team about your blocker for 3 days (B). The backend team built an API assuming wrong params and had to rework it (I)."

Radical Candor Quadrant



Ruinous Empathy (most common EM failure): You care but don't say the hard truth. Engineer suffers long-term.

Obnoxious Aggression: You challenge but don't care. Technically right, relationally damaging.

Manipulative Insincerity: You don't care and don't say it. Toxic.

Radical Candor: You genuinely care AND tell the truth. The goal.

Feedback Delivery Script Templates

Positive reinforcement:

> *"I want to call out something specific. In the sprint planning yesterday, you pushed back on adding three more stories by pointing to the team's actual velocity data. That was exactly right, and it helped us protect the team's capacity. I want you to keep doing that."*

Constructive (first time):

> *"I want to share something I noticed. [SBI]. I'm telling you this because I want you to succeed here, and I think this is holding you back. What's your take on what happened?"*

Escalation (pattern persisting):

> *"We've talked about [behavior] before, on [dates]. I'm concerned because I'm still seeing it. I want to talk about a plan together — not as punishment, but because I need to see change, and I want to support you in making it. If things don't improve by [timeframe], we'll need to involve HR in a more formal process."*

Managing Underperformance

Performance Issue Lifecycle

```
Early Signal → Informal Coaching → Documented Coaching → PIP → Exit or Recovery
    |               |                    |                |         |
  Weeks          Weeks-Months         1-3 months      30-90 days  Outcome
```

Rule: Never surprise an engineer with a PIP. They should have had multiple conversations before it.

PIP Structure

```
## Performance Improvement Plan
## Engineer: [Name] | Manager: [Your Name] | Date: [Date]

### Background
[2-3 sentences describing what led to this PIP, referencing prior coaching conversations]

### Areas of Concern
1. **Code Quality**: In the last 3 sprints, [Name] has introduced 7 P2 bugs that escaped to production. The team
2. **Delivery**: [Name] has missed the agreed-upon sprint commitments in 4 of the last 5 sprints without proacti
3. **Communication**: [Name] has not updated task status in Jira for 3+ days on 6 occasions in the past month, c

### Success Criteria (30-Day Goals)
1. Zero P2+ bugs escaping to production originating from [Name]'s code
2. Communicate blockers within 1 business day of identification
3. Keep Jira task status current with daily updates
4. Achieve code review approval without major revision requests

### Support Plan
- Weekly 1:1 check-ins focused on PIP progress (30 min extra)
- Pair programming sessions with [Senior Engineer] twice per week
- Manager will review every PR before submission during PIP period

### Checkpoints
- Week 2 check-in: [Date]
- Week 4 final review: [Date]

### Consequences
If the success criteria are not met by [Date], [Company] will proceed with a formal separation process.
```

Incident Management Playbook

Severity Definitions

Level	Impact	Response Time	Examples
P0	All users affected, revenue/data loss	Immediate	App crashes on launch, payment failure
P1	Significant % affected, major feature down	< 15 min	Login broken, feed not loading
P2	Partial impact, degraded performance	< 1 hour	Slow load times, image failures
P3	Minor, workaround available	< 4 hours	UI bug, minor feature broken
P4	Negligible impact	Sprint backlog	Cosmetic issues

Incident Response Checklist (iOS-Specific)

Immediate (0–15 min):
- [ ] Acknowledge the incident in incident channel
- [ ] Check Crashlytics / Sentry for crash rate trend
- [ ] Check App Store Connect for user reviews spiking
- [ ] Check server error rates (is it backend or iOS?)
- [ ] Identify affected iOS version range and device types
- [ ] Assign Incident Commander (IC)

Mitigation (15–60 min):
- [ ] If crash in new release: initiate App Store version halt (if phased release)
- [ ] If backend-triggered: coordinate with backend team
- [ ] Enable/disable feature flag if applicable
- [ ] Push emergency fix if code issue — expedited review process?
- [ ] Communicate status to stakeholders every 15-30 min

Resolution:
- [ ] Confirm metrics returning to normal
- [ ] Send resolution notice to stakeholders
- [ ] Schedule post-mortem within 5 business days

Blameless Post-Mortem Template

```
## Incident Post-Mortem
### Incident: [Name/ID]
### Date: [Date] | Severity: P[N]
### Duration: [Start] – [End] | Total Impact: [X] hours / [Y]% of users

---

### Timeline
| Time | Event |
|---|---|
| 14:03 | First alert triggered — crash rate exceeded 2% |
| 14:07 | On-call engineer acknowledged |
| 14:22 | Root cause identified: nil force-unwrap in payment parser |
| 14:40 | Fix deployed via feature flag |
| 15:10 | Crash rate returned to baseline |

---

### Impact
- Users affected: ~120,000 (12% of DAU)
- Failed payment attempts: ~8,400
- Estimated revenue impact: $42,000
- App Store rating drop: 4.6 → 4.1 (recovered in 2 weeks)

---

### Root Cause
A force-unwrap `!` on an optional `currencyCode` field in the payment response parser. The backend team deployed

---

### Why Did This Happen? (Five Whys)
1. Why did the app crash? → Force-unwrap on nil value
2. Why was there a force-unwrap? → Developer assumed field always present per contract
3. Why was assumption wrong? → Backend team made a breaking API change without notice
4. Why wasn't this caught in testing? → API contract testing was not in place
5. Why no API contract testing? → Never prioritized in the roadmap

---

### What Went Well
- On-call response was fast (4 min acknowledgement)
- Feature flag rollback was executed cleanly
- Cross-team communication was effective

---

### Action Items
| Action | Owner | Due Date |
|---|---|---|
| Replace all force-unwrap in network models with safe handling | iOS Team | 1 week |
| Implement Pact contract testing for payment API | iOS + Backend | 3 weeks |
| Add crash rate alert threshold for payment flow | Platform | 1 week |
| Establish breaking change communication protocol with backend | EM | 2 weeks |
| Add non-US payment coverage to UI test suite | QA | 2 weeks |
```

Team Health & Culture

Team Health Survey Questions (Run Quarterly)

Rate 1-5 (Strongly Disagree → Strongly Agree):

Psychological Safety
1. I feel safe to speak up about problems and concerns on this team.
2. I can take risks on this team without feeling afraid of failure.
3. Members of this team never reject others for being different.

Clarity
4. I understand what's expected of me in my role.
5. I know what our team's goals are for this quarter.
6. I understand how my work connects to the company's mission.

Autonomy
7. I have enough autonomy to make decisions about my work.
8. My manager trusts me to do my job without micromanaging.

Growth
9. I have opportunities to learn and grow at this company.
10. My manager actively supports my career development.

Collaboration
11. People on this team cooperate to get things done.
12. We have effective processes for working together.

Recognition
13. My contributions are recognized and valued.
14. I know what it takes to grow and advance on this team.

Overall
15. I would recommend this team to a friend as a great place to work.

Interpreting Results

- **4.0–5.0**: Healthy — focus on maintenance and growth
- **3.0–3.9**: Attention needed — investigate specific low areas
- **Below 3.0**: Critical — immediate action required, leadership escalation

Building Psychological Safety (Practical Actions)

1. **Model vulnerability**: "I made a mistake in that decision. Here's what I learned."
2. **Publicly praise learning, not just results**: "Sarah tried something that didn't work, but we all learned X from it."
3. **Punish silence, not failure**: Ask quiet people for their view; reward dissent.
4. **Separate investigation from blame in incidents**: Post-mortems are about systems, not people.

5. **React to bad news graciously**: If you react with anger to problems being surfaced, people will stop surfacing them.

Working With Your Manager

Managing Up: What Your Manager Needs From You

1. **No surprises**: Surface problems early. "I think the Q3 launch is at risk because X" is always better than a missed launch.
2. **Framed asks**: Don't bring problems without framing your proposed solution.
3. **Visibility**: Keep them informed at the right level of detail (not too deep, not too abstract).
4. **Your priorities**: They can't calibrate your headcount or resources if they don't know what matters to you.
5. **Team wins**: Make your team's work visible to leadership.

Skip-Level Relationship

Your skip-level (your manager's manager) should:
- Know who your strong performers are
- Know your team's key initiatives
- Trust that your team is healthy and executing

How to build the relationship:
- Ask your manager if occasional skip-levels are appropriate
- Send brief monthly written updates up the chain
- Invite skip-level to relevant demos

Disagreeing With Your Manager

When to push back:
- You have data or context they don't
- The decision affects your team's safety, morale, or ethics
- You have domain expertise that changes the analysis

How to push back well:
1. Understand their reasoning first (don't assume)
2. Present your disagreement privately, not publicly
3. Use data, not opinion
4. Accept the final decision if overruled — disagree and commit
5. Know when to escalate (rare): ethics violations, legal concerns

---

Communication Templates

Project Status Update (Weekly)

```
## iOS Team Status Update — Week of [Date]

### 🟢 On Track
- [Feature A]: On track for [Date] release. 80% complete.
- [Feature B]: Design finalized, implementation started.

### 🟡 At Risk
- [Feature C]: Backend API delay may impact our timeline. Mitigation: we're parallelizing UI work with mock data

### 🔴 Off Track
- [Feature D]: Descoped from Q3 to Q4. Root cause: underestimated complexity of offline sync. Stakeholders align

### Key Wins This Week
- Crash-free rate improved from 99.1% → 99.6% after the memory leak fix
- Hired [Name] — starting [Date]

### Needs Attention / Decisions Required
- Need PM alignment on [Feature E] priority before sprint planning Monday

### Key Metrics
| Metric | Current | Target | Trend |
|---|---|---|---|
| Crash-free rate | 99.6% | >99.5% | ↑ |
| App launch time (P50) | 1.2s | <1.5s | → |
| Build time | 4.2 min | <5 min | → |
| Sprint velocity | 42 pts | 40 pts | ↑ |
```

Escalation Message Template

```
**ESCALATION: [Brief Description]**

**Current Impact**: [What is happening, how many users affected, business impact]

**Root Cause**: [Known/unknown — what you know so far]

**Actions Taken**: [What has been done]

**What I Need**: [Specific ask — decision, resource, approval, visibility]

**Timeline**: [When decision is needed]

**Owner**: [Who is responsible for next action]
```

Mobile System Design for iOS Engineering Managers

> *A complete guide to answering mobile system design interview questions and leading system design discussions as an EM.*

The Mobile System Design Interview Framework

Unlike backend system design (which focuses on servers, databases, and distributed systems), mobile system design focuses on the **client application**. Interviewers want to see how you think about the user experience, client-side architecture, offline behavior, and performance.

Framework: RADIO

Use this acronym to structure your answer:

Letter	Step	Focus
R	Requirements	Functional + non-functional
A	Architecture	High-level components
D	Data model	Local + remote data structures
I	Interface	API design, data contracts
O	Optimization	Performance, caching, offline

System Design 1: News Feed App (Instagram-like)

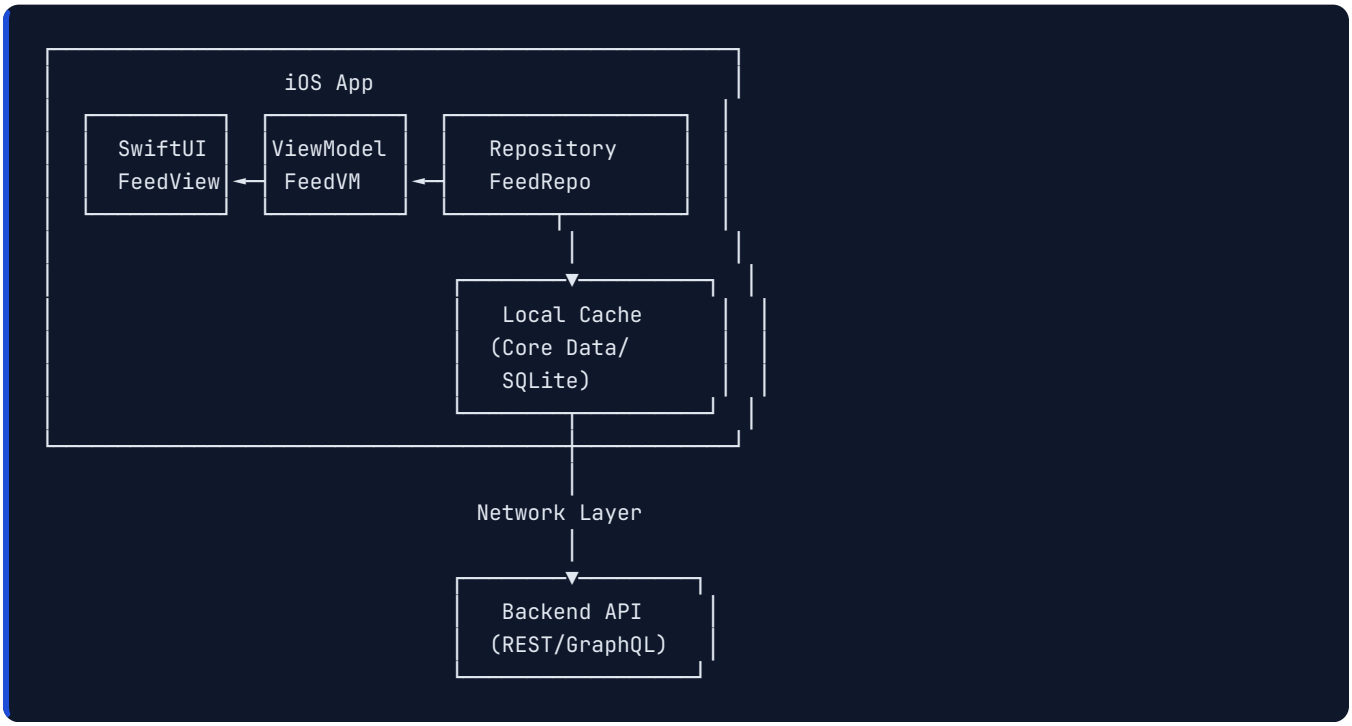
Requirements Clarification

Functional:
- Users can scroll through a feed of posts (images + text)
- Users can like and comment on posts
- Feed updates in near-real-time when online
- Read-only offline support (view last synced feed)

Non-Functional:
- Smooth scrolling at 60fps
- Fast initial load (< 2 seconds)

- Works on slow 3G networks
- Supports iOS 15+

High-Level Architecture



Data Model

Local (Core Data entities):

```
// Post entity
@objc(Post)
class Post: NSManagedObject {
    @NSManaged var postId: String
    @NSManaged var authorId: String
    @NSManaged var authorName: String
    @NSManaged var imageURL: String
    @NSManaged var caption: String
    @NSManaged var likeCount: Int32
    @NSManaged var commentCount: Int32
    @NSManaged var createdAt: Date
    @NSManaged var isLikedByCurrentUser: Bool
    @NSManaged var fetchedAt: Date  // for cache invalidation
}

// Image cache metadata
@objc(CachedImage)
class CachedImage: NSManagedObject {
    @NSManaged var url: String
    @NSManaged var localPath: String
    @NSManaged var cachedAt: Date
    @NSManaged var fileSize: Int64
}
```

Remote (API response model):

```
struct PostResponse: Decodable {
    let postId: String
    let author: AuthorResponse
    let media: MediaResponse
    let engagement: EngagementResponse
    let createdAt: Date

    struct AuthorResponse: Decodable {
        let userId: String
        let displayName: String
        let avatarURL: URL
    }

    struct MediaResponse: Decodable {
        let imageURL: URL
        let width: Int
        let height: Int
        let blurHash: String  // Low-res placeholder
    }

    struct EngagementResponse: Decodable {
        let likeCount: Int
        let commentCount: Int
        let isLikedByCurrentUser: Bool
    }
}
```

API Design

Feed Pagination (Cursor-based, not offset):

```
GET /v1/feed?cursor=<cursor_token>&limit=20

Response:
{
  "posts": [...],
  "nextCursor": "eyJpZCI6IjEyMzQ1NiIsInRzIjoiMjAyNC0...",
  "hasMore": true
}
```

Why cursor over offset?

- Offset is unstable: if new posts inserted, pages shift
- Cursor is stable: consistent pagination even with live insertions
- Better for infinite scroll pattern

Optimistic Like:

```
POST /v1/posts/{postId}/likes
Body: { "userId": "current_user_id" }

// iOS sends immediately, updates local state optimistically
// Rollback UI if request fails
```

Feed Loading Strategy

```
class FeedViewModel: ObservableObject {
    @Published var posts: [PostViewModel] = []
    @Published var isLoadingInitial = false
    @Published var isLoadingMore = false

    private var nextCursor: String?
    private let repository: FeedRepository

    func loadInitialFeed() async {
        isLoadingInitial = true
        defer { isLoadingInitial = false }

        // 1. Show cached posts immediately (offline-first)
        let cached = await repository.fetchCachedPosts(limit: 20)
        posts = cached.map(PostViewModel.init)

        // 2. Fetch fresh from network in background
        do {
            let fresh = try await repository.fetchFeedFromNetwork(cursor: nil)
            posts = fresh.posts.map(PostViewModel.init)
            nextCursor = fresh.nextCursor

            // 3. Update cache
            await repository.cachePosts(fresh.posts)
        } catch {
            // Network failed — cached posts remain visible
            // Show subtle "offline" indicator
        }
    }

    func loadMoreIfNeeded(currentPost: PostViewModel) async {
        guard currentPost.id == posts.suffix(5).first?.id,
              !isLoadingMore,
              let cursor = nextCursor else { return }

        isLoadingMore = true
        defer { isLoadingMore = false }

        let more = try? await repository.fetchFeedFromNetwork(cursor: cursor)
        if let more {
            posts.append(contentsOf: more.posts.map(PostViewModel.init))
            nextCursor = more.nextCursor
        }
    }
}
```

Image Loading & Caching

```
// Three-tier caching: Memory → Disk → Network
class ImageCache {
    private let memoryCache = NSCache<NSString, UIImage>()
    private let diskCache = DiskImageCache()

    func load(url: URL) async throws → UIImage {
        let key = url.absoluteString as NSString

        // 1. Memory cache — instant
        if let cached = memoryCache.object(forKey: key) {
            return cached
        }

        // 2. Disk cache — fast
        if let diskCached = await diskCache.image(for: url) {
            memoryCache.setObject(diskCached, forKey: key)
            return diskCached
        }

        // 3. Network — slowest
        let (data, _) = try await URLSession.shared.data(from: url)
        guard let image = UIImage(data: data) else {
            throw ImageError.invalidData
        }

        // Store in both caches
        memoryCache.setObject(image, forKey: key)
        await diskCache.store(image, for: url)

        return image
    }
}
```

BlurHash for progressive loading:

```
// Show blurry placeholder while full image loads
struct PostImageView: View {
    let post: PostViewModel
    @State private var image: UIImage?

    var body: some View {
        ZStack {
            // Instant blurry placeholder from BlurHash
            BlurHashImage(blurHash: post.blurHash, size: .init(width: 32, height: 32))
                .opacity(image == nil ? 1 : 0)

            // Full image when loaded
            if let image {
                Image(uiImage: image)
                    .resizable()
                    .scaledToFill()
                    .transition(.opacity)
            }
        }
        .task {
            image = try? await ImageCache.shared.load(url: post.imageURL)
        }
    }
}
```

Performance Optimizations

1. **Prefetching**: Pre-load images for off-screen cells

```
extension FeedViewController: UICollectionViewDataSourcePrefetching {
    func collectionView(_ collectionView: UICollectionView,
                        prefetchItemsAt indexPaths: [IndexPath]) {
        let urls = indexPaths.compactMap { posts[$0.row].imageURL }
        ImagePrefetcher.shared.startPrefetching(urls: urls)
    }
}
```

1. **Cell height caching**: Avoid repeated layout calculations
2. **Diffable data source**: Animate only changed cells
3. **Image resizing**: Download image at display size, not full resolution
4. **Background decoding**: Decode images off main thread

System Design 2: Offline-First Chat App

Architecture Decision: Offline-First

```
User sends message
        ↓
Save to local DB immediately (status: "sending")
        ↓
Update UI optimistically (message appears instantly)
        ↓
Send to server in background
        ↓
Server ACK received → update status to "sent"
Server delivery confirmed → update to "delivered"
Recipient reads → update to "read"
```

If send fails:

- Keep message in local DB with status "failed"
- Show retry button
- Background retry queue attempts periodically

Message Queue Architecture

```
actor MessageQueue {
    private var pendingMessages: [PendingMessage] = []
    private var isSending = false

    func enqueue(_ message: PendingMessage) async {
        pendingMessages.append(message)
        await flush()
    }

    private func flush() async {
        guard !isSending, !pendingMessages.isEmpty else { return }
        isSending = true
        defer { isSending = false }

        while let message = pendingMessages.first {
            do {
                try await networkClient.sendMessage(message)
                pendingMessages.removeFirst()
                await updateLocalStatus(message.id, status: .sent)
            } catch {
                // Exponential backoff before retry
                await Task.sleep(nanoseconds: 2_000_000_000)
                // Will retry on next flush call
                break
            }
        }
    }
}
```

WebSocket Management

```
class WebSocketManager {
    private var webSocketTask: URLSessionWebSocketTask?
    private let session: URLSession
    private var reconnectAttempts = 0
    private let maxReconnectAttempts = 10

    func connect(token: String) {
        var request = URLRequest(url: wsURL)
        request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
        webSocketTask = session.webSocketTask(with: request)
        webSocketTask?.resume()
        receiveMessages()
        startPingTimer()
    }

    private func receiveMessages() {
        webSocketTask?.receive { [weak self] result in
            switch result {
            case .success(let message):
                self?.handle(message)
                self?.receiveMessages() // Continue listening
            case .failure:
                self?.scheduleReconnect()
            }
        }
    }

    private func scheduleReconnect() {
        guard reconnectAttempts < maxReconnectAttempts else { return }
        let delay = pow(2.0, Double(reconnectAttempts)) // Exponential backoff
        reconnectAttempts += 1
        DispatchQueue.main.asyncAfter(deadline: .now() + delay) { [weak self] in
            self?.connect(token: ...)
        }
    }

    private func startPingTimer() {
        Timer.scheduledTimer(withTimeInterval: 30, repeats: true) { [weak self] _ in
            self?.webSocketTask?.sendPing { _ in }
        }
    }
}
```

System Design 3: iOS SDK for Analytics

Design Goals for an SDK

- **Zero impact on host app performance** (no main thread blocking)
- **Minimal binary size increase** (< 500KB)
- **Simple public API** (one-line event tracking)
- **Reliable delivery** (retry on failure, persistence across app restarts)

- **Batching** (don't send every event individually)

Public API Design

```
// Simple, ergonomic public API
public final class Analytics {
    public static let shared = Analytics()

    private init() {}

    public func configure(apiKey: String, options: Options = .default) {
        AnalyticsEngine.shared.configure(apiKey: apiKey, options: options)
    }

    public func track(event: String, properties: [String: Any] = [:]) {
        AnalyticsEngine.shared.track(
            Event(name: event, properties: properties, timestamp: Date())
        )
    }

    public func identify(userId: String, traits: [String: Any] = [:]) {
        AnalyticsEngine.shared.identify(userId: userId, traits: traits)
    }

    public func screen(_ name: String, properties: [String: Any] = [:]) {
        track(event: "Screen Viewed", properties: ["screen": name].merging(properties) { $1 })
    }

    public struct Options {
        public var batchSize: Int = 20
        public var flushInterval: TimeInterval = 30
        public var maxQueueSize: Int = 1000
        public var endpoint: URL = URL(string: "https://api.analytics.com/v1/batch")!

        public static let `default` = Options()
    }
}
```

Internal Engine Design

```
actor AnalyticsEngine {
    static let shared = AnalyticsEngine()

    private var eventQueue: [Event] = []
    private var flushTask: Task<Void, Never>?
    private var configuration: Analytics.Options = .default
    private let persistence: EventPersistence
    private let networkClient: AnalyticsNetworkClient

    func track(_ event: Event) {
        // Limit queue size to prevent memory issues
        guard eventQueue.count < configuration.maxQueueSize else { return }

        eventQueue.append(event)
        persistence.append(event) // Persist to survive app restart

        if eventQueue.count ≥ configuration.batchSize {
            Task { await flush() }
        }
    }

    func flush() async {
        guard !eventQueue.isEmpty else { return }

        let batch = Array(eventQueue.prefix(configuration.batchSize))

        do {
            try await networkClient.sendBatch(batch)
            eventQueue.removeFirst(min(batch.count, eventQueue.count))
            persistence.remove(events: batch)
        } catch {
            // Keep in queue for retry — persistence ensures survival
        }
    }

    private func startFlushTimer() {
        flushTask?.cancel()
        flushTask = Task {
            while !Task.isCancelled {
                try? await Task.sleep(nanoseconds: UInt64(configuration.flushInterval * 1_000_000_000))
                await flush()
            }
        }
    }
}
```

System Design 4: App Modular Architecture

Monolith → Modular Migration

Starting state: Single app target with everything in it

Problem symptoms:
- Build time > 10 minutes
- Any change triggers full recompile
- Engineers step on each other's code
- Impossible to share code with extensions

Target state: Feature modules with clear ownership

Migration Strategy: Strangler Fig

```
Phase 1: Extract shared utilities (lowest risk)
  Month 1-2: Networking, Analytics, DesignSystem, Extensions

Phase 2: Extract core business logic
  Month 3-4: Auth module, UserProfile module

Phase 3: Extract feature modules (one per sprint)
  Month 5+: CheckoutFeature, SearchFeature, HomeFeature, etc.
```

Module Interface Design

```
// Public interface of a feature module
// Other modules only see this, not implementation details

public protocol CheckoutFeatureInterface {
    func startCheckout(cartId: String) → UIViewController
    func continueCheckout(orderId: String) → UIViewController
}

// Internal implementation — hidden from other modules
final class CheckoutCoordinator: CheckoutFeatureInterface {
    private let dependencies: CheckoutDependencies

    public func startCheckout(cartId: String) → UIViewController {
        let vc = CheckoutStartViewController(cartId: cartId,
                                             dependencies: dependencies)
        return UINavigationController(rootViewController: vc)
    }
}
```

Build Time Impact of Modularization

With a single monolith target:
- Any file change → recompile everything → 10+ min build

With modules (type-checked incrementally):
- Change in `FeatureCheckout` → recompile only `FeatureCheckout` + `App` → 1-2 min
- Change in `DesignSystem` → recompile `DesignSystem` + dependents → 3-5 min
- Change in `Networking` → full rebuild (everyone depends on it) — keep this module stable

---

System Design Interview: How to Stand Out

Signals Interviewers Look For

Good Signal	How to Show It
User empathy	"Users on slow networks will see..."
iOS platform depth	Reference real APIs (NSCache, URLSession, NWPathMonitor)
Trade-off awareness	"Cursor pagination beats offset because..."
Scale thinking	"At 10M users, this cache eviction policy..."
Testability	"This architecture makes it easy to mock the repository..."
Monitoring	"I'd track these metrics to detect regressions..."

Common Mistakes to Avoid

1. **Jumping to code too fast**: Spend 5 min on requirements first
2. **Ignoring offline**: Almost every mobile design needs offline consideration
3. **Over-engineering**: Match complexity to stated requirements
4. **Only happy path**: Address error states, retry, edge cases
5. **Ignoring performance**: Memory, battery, network are always relevant on mobile
6. **Not drawing**: Visual aids massively help in system design — always whiteboard

Template Opening Statement

> *"Before I dive into the design, let me clarify a few things to make sure I'm solving the right problem. [Ask 2-3 clarifying questions]. Based on your answers, here's how I'm thinking about this..."*

---

iOS Architecture for EM: Trade-off Cheat Sheet

Architecture	Best For	Avoid When	Test Difficulty	Code Volume
MVC	Tiny screens, prototypes	Large teams, complex logic	Hard	Low
MVP	Mid-size apps, UIKit	SwiftUI-first	Medium	Medium
MVVM	Most apps, SwiftUI	Tiny apps	Easy	Medium
VIPER	Large teams, strict roles	Solo/small teams	Easiest	Very High
TCA	Complex state machines	Simple CRUD apps	Easiest	High
Clean	Enterprise, multiple platforms	Small apps	Medium-Hard	Very High

The EM's Rule of Architecture Choice

> *"The best architecture is the one your team can understand, maintain, and onboard new engineers into. An architecture that looks beautiful in docs but confuses your team in code is not a good architecture."*

Interview Preparation: Engineering Manager - iOS

> *60 prepared STAR stories, behavioral frameworks, and company-specific tips.*

How the EM Interview Is Different from IC

As an IC, you're interviewed on **what you can do**.
As an EM, you're interviewed on **what your team can do through you**.

The shift:
- **IC**: "Tell me about a hard technical problem you solved."
- **EM**: "Tell me about a time you helped your team solve a hard technical problem."

Your individual contributions matter less. Your **judgment**, **leadership**, and **team outcomes** matter most.

Behavioral Interview Themes & STAR Stories

Theme 1: Technical Leadership

What they're assessing: Can you make sound technical decisions, earn engineer trust, and drive technical quality?

Question: "Tell me about a significant technical decision you drove for your iOS team."

STAR Template:

> ***S**: We had a 3-year-old UIKit codebase with no architecture pattern. Onboarding took engineers 3-4 weeks before they could contribute meaningfully. Bugs were increasing despite growing team size.*
> ***T**: As the new EM, I needed to establish a technical foundation that would scale us from 4 to 12 engineers over the next 18 months.*
> ***A**: I ran a two-week architecture spike with our two most senior engineers. We evaluated MVVM+Coordinator (our team's preference) vs. VIPER (company standard elsewhere) vs. TCA (new but powerful). I facilitated the discussion, synthesized the trade-offs, and wrote the ADR. We chose MVVM+Coordinator because our team understood it, SwiftUI was our direction, and VIPER's verbosity would slow us down. I also made it a team decision — they owned it more because they participated.*

> ***R**: Within 6 months, 4 new features were built on the new pattern. Onboarding dropped from 3-4 weeks to 1.5 weeks. Code review feedback shifted from architecture corrections to logic discussions. Engineer satisfaction increased by 18% on our team health survey.*

Question: "Describe a time you made a technical call that turned out to be wrong. What did you do?"

> ***S**: We decided to build our own in-house image caching library instead of using Kingfisher or SDWebImage. I pushed for this, believing we needed custom behavior for our CDN.*
> ***T**: Six weeks into implementation, we were behind schedule and the library had edge cases we hadn't anticipated.*
> ***A**: I acknowledged to the team that the decision was wrong — I said it explicitly, didn't bury it. We ran a quick analysis: 3 more weeks to reach parity with open source libraries vs. switching to Kingfisher now. I made the call to switch. We held a brief retrospective on the decision-making process to understand how we didn't catch the risks earlier.*
> ***R**: We shipped on time by switching. More importantly, we documented a "build vs. buy" checklist that we've used on 4 subsequent decisions. The team trusted my judgment more after seeing me admit the mistake openly.*

Theme 2: People Management

What they're assessing: Can you grow engineers, handle underperformance, and build strong teams?

Question: "Tell me about a time you helped an underperforming engineer improve."

> ***S**: One of my senior iOS engineers was consistently missing sprint commitments and writing code that frequently required significant rework in code review. This had been happening for about 8 weeks.*
> ***T**: I needed to address this directly without damaging morale or crossing into a formal PIP prematurely.*
> ***A**: I had a frank 1:1 using specific data: "In the last 3 sprints, 4 of your 6 committed stories weren't completed. Here are the code review threads where reviewers asked for significant changes. I want to understand what's going on." I discovered they were going through a difficult personal situation and also felt they were in the wrong part of the codebase — iOS payments was not their strength; they were stronger in UI. I restructured their assignments to play to their strengths, connected them with a therapist resource through our EAP, and set up biweekly check-ins specifically focused on clarity and blockers.*
> ***R**: Within 6 weeks, delivery improved significantly. They've since become one of our best engineers for complex UI/UX features. They told me in a 1:1 later that they almost quit during that period, and the direct-but-caring conversation changed their mind.*

Question: "Tell me about a time you had to let someone go."

> ***S**: After a 90-day PIP, one of my engineers had not met the agreed improvement criteria. They had improved in some areas but the core issue — quality of technical judgment on architecture decisions — was still significantly below the level for their role.*
> ***T**: I needed to execute the separation professionally, compassionately, and legally correctly.*
> ***A**: I worked with HR to prepare the termination documentation and aligned on logistics (severance, benefits). In the meeting, I was direct: "Based on our PIP review, you haven't met the success criteria we agreed on, and we're moving to separate." I acknowledged their positive contributions, thanked them for their effort, and let them ask questions. I ensured they had all the information about severance and COBRA. Afterward, I addressed the team — acknowledging someone had left, that it was a management decision, and that I was available for 1:1 conversations if anyone had concerns.*
> ***T**: The team appreciated the transparency. Within 2 months, we hired a replacement. The team's retrospective feedback on how I handled it was largely positive — they felt the process had been fair.*

Question: "How do you handle a situation where your best engineer wants to leave?"

> ***S**: My tech lead told me she was interviewing at Google. She was the highest performer on the team and the most knowledgeable about our payments module.*
> ***T**: I needed to retain her without making unrealistic promises, or handle the departure gracefully.*
> ***A**: I didn't panic. I asked her to walk me through what was drawing her to Google. It was: larger scale technical problems and a promotion to Staff Engineer. I was honest: "I can't manufacture scale here overnight, but let me understand what specifically about Staff-level work appeals to you." Over 3 conversations, we built a plan: I got her a Staff Engineer title in our next review cycle, gave her ownership of the architecture for our new real-time features (higher complexity, larger scope), and set up bi-annual senior engineer meetings she could present in. I also looped in her skip-level to recognize her impact more publicly.*
> ***R**: She stayed. 18 months later, she's led three major platform initiatives. I also had an honest conversation with myself: if someone as good as her feels constrained, what's that telling me about how I'm managing growth opportunities? I changed how I allocate stretch projects across the team.*

Theme 3: Delivery & Execution

What they're assessing: Can you deliver on time, navigate ambiguity, and manage risk?

Question: "Tell me about a project that was severely at risk. How did you save it?"

> ***S**: Our iOS checkout redesign was 6 weeks from launch when we discovered a critical bug: our payment tokenization was incompatible with the new backend API contract. Fixing it properly would take an estimated 3 weeks — we had 6 weeks but also 4 weeks of remaining feature work.*

> ***T**: We had a firm launch date tied to a marketing campaign already announced publicly. We couldn't move the date.*
> ***A**: I ran an emergency war room with backend and iOS leads. Option A: build a compatibility shim (2 weeks), launch on time, fix properly in the next sprint. Option B: descope 2 features, fix properly, launch the core checkout. I consulted the PM and VP on impact. We chose Option A for the shim + descoped 2 non-critical features. I ran daily standups for the 3-week sprint, escalated a backend dependency that was blocking us (getting a senior backend engineer dedicated to this), and tracked risks daily. We also added an extra QA cycle.*
> ***R**: We launched on time. Conversion rate for the new checkout was 14% better than the previous version. The shim was replaced cleanly 6 weeks later. We updated our API contract review process to catch compatibility issues earlier — this was a process gap we fixed.*

Question: "How do you balance technical debt with feature delivery?"

> ***S**: Our iOS app had accumulated significant technical debt over 2 years — no tests, inconsistent networking layer, outdated dependencies. PM wanted maximum feature velocity.*
> ***T**: I needed to address the debt without grinding feature work to a halt or creating conflict with the product team.*
> ***A**: First, I quantified the cost of the debt: we tracked bugs caused by the debt (35% of all bugs), developer velocity impact (estimated 20% slower than a clean codebase), and on-call burden (40% of incidents related to the old networking layer). I presented this as a business case to the PM: "We're spending 20% of our capacity dealing with debt consequences. If we invest 15% of capacity in structured debt reduction for 2 quarters, I project 10-15% faster velocity by Q3." I proposed a specific plan: 15% of sprint capacity permanently reserved for debt, with a focused 6-week "tech reset" sprint for the worst offenders. We agreed on this.*
> ***R**: 6 months later, bug rate dropped by 40%, on-call incidents from old networking dropped by 60%, and we actually shipped 2 more features in Q4 than Q3 despite the investment. The PM became one of the strongest advocates for our debt budget after seeing the data.*

Theme 4: Conflict Resolution

What they're assessing: Can you navigate interpersonal and organizational conflict productively?

Question: "Tell me about a conflict between two engineers on your team. How did you resolve it?"

> ***S**: Two senior iOS engineers had a persistent conflict over architectural direction. Engineer A preferred MVVM, Engineer B was pushing for Clean Architecture with VIPER. It was affecting code reviews (both were blocking each other's PRs unnecessarily) and team meetings were becoming tense.*
> ***T**: I needed to resolve the technical disagreement and the interpersonal dynamic before it became toxic.*

> ***A**: I met with each of them separately first to understand both technical and personal perspectives. The technical disagreement was real — both had valid points. But underneath, Engineer B felt his experience wasn't being respected (he'd worked at a larger company with stricter patterns). I facilitated a structured architecture decision session: I wrote the evaluation criteria in advance (testability, onboarding speed, consistency with platform trends), had each of them present their proposal against those criteria, and invited two external engineers to participate for a neutral perspective. I then made the final call (MVVM with some Clean Architecture principles for business logic), explaining my reasoning clearly. I also set up a regular "architecture office hours" session so Engineer B had a channel to influence technical direction constructively.*
> ***R**: The decision was clear and owned. Both engineers accepted it. Engineer B later told me the structured process helped him feel heard even though he didn't "win." They now collaborate effectively and even pair on complex problems.*

Theme 5: Strategy & Vision

What they're assessing: Can you think beyond quarters and set direction?

Question: "How have you built a long-term technical roadmap for your iOS team?"

> ***S**: When I joined, the iOS team had no documented technical strategy. Engineers made decisions ad hoc, creating inconsistencies and accumulating debt.*
> ***T**: I needed to create a 12-18 month iOS platform strategy that aligned with business goals and could get engineer buy-in.*
> ***A**: I ran a two-week strategy process: (1) Audit: reviewed the codebase, interviewed all engineers and key stakeholders, reviewed App Store reviews and crash data. (2) Vision: drafted a "North Star" for the iOS platform in 18 months — modular, tested, accessible, performant. (3) Gap analysis: mapped current state vs. target state with specific metrics. (4) Roadmap: phased plan with quarterly milestones, assigned owners. (5) Socialization: presented to engineers for input, then to leadership for alignment and resources. I wrote it as a live document (Notion) so it could be updated as strategy evolved.*
> ***R**: Within 3 months, every engineer could articulate our platform direction. Decision-making improved — engineers started referencing the strategy in code reviews and design discussions. We got headcount approval for 2 additional engineers because the strategy demonstrated a clear return on investment.*

Company-Specific Preparation

Apple

Culture signals to demonstrate:
- Extreme attention to quality and detail

- Deep platform and HIG knowledge
- Confidentiality mindset (Apple doesn't talk externally)
- Long-term thinking over short-term optimization

Prepare to discuss:
- Your experience with Apple-specific APIs (Core Animation, SwiftUI, Core Data)
- How you've ensured quality at every stage of development
- Your philosophy on user privacy and data handling

Google

Culture signals:
- Structured, data-driven decision making
- SMART goals (OKRs heavily used)
- Career development and growth mindset
- Large-scale thinking

Prepare to discuss:
- How you've used data to drive decisions
- Your experience setting and measuring OKRs
- How you've scaled teams and systems
- Your approach to performance management

Meta (Facebook/Instagram)

Culture signals:
- Move fast (velocity matters)
- Think at scale (millions of users)
- Impact-first (results over process)
- Cross-functional collaboration

Prepare to discuss:
- Specific measurable impact of your work
- Examples of moving quickly without breaking things
- Handling rapid growth and scale

Amazon (Leadership Principles)

Amazon interviews map directly to their 16 Leadership Principles. Prepare one STAR story for each:

Principle	Story Focus
Customer Obsession	Time you made a product decision from the user's perspective
Ownership	Time you went beyond your role to fix a problem
Invent and Simplify	Technical innovation that simplified a process
Are Right, A Lot	Time you made an unpopular decision that turned out correct
Learn and Be Curious	Technology or skill you learned recently
Hire and Develop the Best	Best hire you've made; how you developed someone
Insist on the Highest Standards	Time you raised the quality bar
Think Big	Long-term vision you set for your team
Bias for Action	Time you made a fast decision with incomplete info
Frugality	Time you did more with less
Earn Trust	Time you rebuilt trust after a mistake
Dive Deep	Technical deep-dive that revealed a root cause
Have Backbone; Disagree and Commit	Time you pushed back on leadership
Deliver Results	Project delivered against all odds
Strive to be Earth's Best Employer	Specific thing you did for team well-being
Success and Scale Bring Broad Responsibility	Ethical or inclusion consideration in engineering

Technical Interview Preparation

iOS Coding Screen — What to Practice

Companies screen for **real-world iOS problems**, not LeetCode-style algorithms (usually). Practice:

1. **Build a simple list feature** — networking, decoding, display
2. **Implement a cache** — protocol-based, with expiration
3. **Debug a retain cycle** — identify and fix in code
4. **Design a protocol** — for a repository or service
5. **Write a unit test** — for a ViewModel with a mock dependency
6. **Implement async/await** — convert a completion-handler function

Swift Coding Warm-Up (30 min/day for 2 weeks)

```
// Practice these patterns daily:

// 1. Generic constraints
func findFirst<T: Equatable>(_ value: T, in array: [T]) → Int? {
    array.firstIndex(of: value)
}

// 2. Protocol with associated type
protocol Repository {
    associatedtype Model
    func fetch(id: String) async throws → Model
    func save(_ model: Model) async throws
}

// 3. Result type usage
func loadConfig() → Result<Config, ConfigError> {
    guard let url = Bundle.main.url(forResource: "config", withExtension: "json") else {
        return .failure(.missingFile)
    }
    // ...
}

// 4. Custom property wrapper
@propertyWrapper
struct Clamped<T: Comparable> {
    private var value: T
    let range: ClosedRange<T>

    var wrappedValue: T {
        get { value }
        set { value = min(max(newValue, range.lowerBound), range.upperBound) }
    }

    init(wrappedValue: T, _ range: ClosedRange<T>) {
        self.range = range
        self.value = min(max(wrappedValue, range.lowerBound), range.upperBound)
    }
}
```

Questions to Ask Your Interviewer

These signal strategic thinking and genuine interest:

For the hiring manager:

- "What does success look like for this role in the first 6 months?"
- "What's the biggest technical challenge the iOS team is facing right now?"
- "How is technical debt currently managed and prioritized?"
- "What does the relationship between the EM and PM look like on this team?"

For the team:

- "What's one thing you wish was different about how the team works?"

- "What made you join this team, and what keeps you here?"
- "How does the team approach disagreement on technical decisions?"

For skip-level:
- "What are the most important leadership traits you're looking for in this EM?"
- "How does engineering leadership fit into the broader company strategy?"
- "What opportunities do you see for this team over the next 2 years?"

Day-Of Interview Tips

Before the Interview

- Review the company's latest App Store release notes (shows you care about their product)
- Look up recent engineering blog posts from the company
- Review the job description and prepare 3 specific examples that map to each key requirement
- Prepare your "Why this company?" answer — make it specific, not generic

During the Interview

- **Listen more than you talk** initially — let them finish their question
- **Think out loud** — interviewers want to hear your reasoning process
- **Ask clarifying questions** — especially in system design
- **Acknowledge when you don't know** — "I haven't worked with X directly, but my approach would be..."
- **Pause before answering** — 5-10 seconds of thinking beats a rushed answer
- **Quantify impact** — "we reduced crash rate" is weak; "we reduced crash rate from 2.1% to 0.4%" is strong

Handling "I Don't Know"

Never bluff. Instead:

> *"I haven't worked with that specific technology. What I would do is [reasoning process + adjacent knowledge]. Is that the right direction, or would you like to tell me more about the context?"*

Ending Each Interview

> *"Is there anything in our conversation where you'd like me to go deeper, or anything that gave you pause that I could address?"*

This shows confidence and gives you a chance to fix a weak answer.

Compensation Negotiation

Know Your Worth

Research channels:
- Levels.fyi (most accurate for total compensation at tech companies)
- LinkedIn Salary
- Glassdoor
- Ask in communities: Rands Leadership Slack, iOS Dev Happy Hour

Total Compensation Components

Component	Notes
Base salary	Fixed annual
Equity (RSUs/options)	Vesting schedule (typically 4-year, 1-year cliff)
Sign-on bonus	Often negotiable; fills equity vesting gap
Annual bonus	Target % of base
Benefits	Healthcare, 401k match, PTO, parental leave

Negotiation Framework

1. **Don't anchor first**: "I'm flexible — what's the range for this role?"
2. **Anchor high if forced**: Give a number 15-20% above your target
3. **Negotiate total comp**, not just base
4. **Use competing offers** as leverage (real leverage, not bluffing)
5. **Ask for time**: "Can I have 48 hours to review this with my family?"

What to Say

> *"I'm very excited about this opportunity. Based on my research on Levels.fyi and conversations with peers, I was expecting total compensation in the range of [$X-Y]. Is there flexibility to reach that range?"*

Skill-Building Action Plan: Engineering Manager - iOS

> *A structured, time-phased plan to build every skill needed to land and excel in an EM - iOS role.*

Self-Assessment: Where Are You Today?

Rate yourself honestly on each area (1-5):

iOS Technical Skills

Skill	1 (Beginner)	3 (Intermediate)	5 (Expert)	Your Score
Swift language (advanced)	Can't explain ARC	Comfortable with generics, protocols	Deep knowledge of concurrency, macros	___
UIKit	Can't build a list	Builds features independently	Expert in collection views, animation, perf	___
SwiftUI	Never used it	Built simple screens	Built complex apps, knows internals	___
iOS Architecture	Knows MVC	Implements MVVM	Chose arch for multiple real projects	___
Concurrency	Uses DispatchQueue.main	Understands GCD	Deep knowledge of Swift Concurrency, actors	___
Testing	No tests	Some unit tests	TDD, 80%+ coverage, UI tests, mocks	___
Performance	Not profiled apps	Used Instruments once	Optimized launch, scroll, memory in prod	___
Networking	Makes URLSession calls	Designed a network layer	Production-grade layer with retry, caching	___
CI/CD	Never set up	Used existing pipelines	Set up Fastlane + Xcode Cloud from scratch	___
Security	Basic HTTPS	Keychain usage	Certificate pinning, CryptoKit, OAuth/PKCE	___

Management Skills

Skill	1 (No Experience)	3 (Some Experience)	5 (Expert)	Your Score
1:1s	Never done	Running 1:1s	Coaching, relationship-building, career convos	___
Performance management	Never done	Given feedback	Full PIP cycle, reviews, promotions	___
Hiring	Never interviewed	Participated in interviews	Owned loop, designed process, made offers	___
Goal setting (OKRs)	Never done	Written OKRs	Aligned team to company strategy via OKRs	___
Conflict resolution	Avoid conflict	Address directly	Skilled facilitator, resolved team conflicts	___
Technical strategy	Never done	Contributed to strategy	Authored and presented platform strategy	___
Stakeholder management	Individual contributor	Some PM/Design interaction	Owned relationships across multiple functions	___
Incident management	Not on call	Participated in incidents	Led P0 response, post-mortems, process design	___
Delivery management	No tracking	Tracks sprint	Manages roadmap, risk, scope negotiation	___
Hiring	Never hired	1-2 hires	Built team, pipeline, interview process	___

Scoring Guide

Total Score (sum of all 20)	Status
80-100	Ready for senior EM interviews
60-79	Ready for EM interviews, gaps to address
40-59	Needs 6-12 months of deliberate practice
Below 40	Needs 12-18+ months; consider Tech Lead first

6-Month Skill-Building Plan

Month 1: iOS Technical Foundation

Goal: Shore up any technical gaps. You must be credible with engineers.

Week 1-2: Swift & Concurrency
- [ ] Read the Swift book chapters on generics, protocols, concurrency
- [ ] Complete the WWDC sessions: "Meet async/await in Swift", "Protect mutable state with Swift actors"
- [ ] Build a small project using async/await + actors (e.g., a weather app)
- [ ] Watch "Eliminate data races using Swift Concurrency" (WWDC)

Week 3-4: Architecture
- [ ] Read "Advanced iOS App Architecture" by Rene Cacheaux (Ray Wenderlich)
- [ ] Refactor a personal project from MVC to MVVM+Coordinator
- [ ] Study TCA: read Point-Free episodes on The Composable Architecture
- [ ] Write an ADR for your architecture decision

Deliverable: A GitHub project demonstrating MVVM+Coordinator with async/await networking, Core Data, and 70%+ test coverage.

Month 2: iOS Platform & DevOps

Goal: Be able to set up and manage iOS platform infrastructure.

Week 1-2: Testing
- [ ] Achieve 80% unit test coverage on your Month 1 project
- [ ] Add UI tests for the critical user flow
- [ ] Add snapshot tests (swift-snapshot-testing) for key screens
- [ ] Set up test coverage reporting

Week 3-4: CI/CD & DevOps
- [ ] Set up Fastlane for your project (scan, gym, deliver)
- [ ] Set up GitHub Actions to run tests on every PR
- [ ] Configure code signing with `fastlane match` (if Apple Developer account available)
- [ ] Integrate Crashlytics for crash monitoring

Deliverable: GitHub project with full CI/CD pipeline running on every commit.

Month 3: Performance & Advanced iOS

Goal: Develop the profiling and optimization skills engineers expect from a strong EM.

Week 1-2: Instruments & Performance
- [ ] Profile your Month 1 app with Time Profiler — find and fix one hotspot
- [ ] Profile with Allocations — find a memory inefficiency
- [ ] Implement image caching with 3-tier strategy (memory, disk, network)
- [ ] Measure and optimize app launch time

Week 3-4: Security & Accessibility
- [ ] Implement Keychain storage for a credential
- [ ] Implement biometric authentication (TouchID/FaceID) with fallback
- [ ] Add accessibility labels and VoiceOver support to your app
- [ ] Run the accessibility inspector on your app — fix 3 issues

Deliverable: A blog post or internal document describing what you found and fixed in each profiling session.

Month 4: Management Fundamentals

Goal: Build the people management toolkit.

Week 1-2: 1:1s and Feedback
- [ ] Practice 1:1 templates with a peer (role-play)
- [ ] Practice giving SBI feedback (3 scenarios) — record yourself, review
- [ ] Read "Radical Candor" (Kim Scott) — summarize the key model
- [ ] Read "The Manager's Path" (Camille Fournier) — take notes on EM chapter

Week 3-4: Hiring and Performance
- [ ] Design a complete iOS engineer interview loop (4 sessions, rubrics for each)
- [ ] Write 5 sample performance reviews for hypothetical engineers at different levels
- [ ] Write a sample PIP for a hypothetical underperforming engineer
- [ ] Write sample OKRs for a hypothetical iOS team for Q1

Deliverable: A "Management Toolkit" document with your 1:1 template, feedback frameworks, hiring rubric, and OKR examples.

Month 5: Strategy & Leadership

Goal: Think and communicate at the EM level.

Week 1-2: Technical Strategy
- [ ] Write a 12-month iOS platform strategy for a hypothetical company (3-5 pages)
- [ ] Create a Technology Radar for iOS tools/libraries
- [ ] Write 5 ADRs for real or hypothetical iOS decisions
- [ ] Read "An Elegant Puzzle" (Will Larson) — focus on systems and processes chapters

Week 3-4: Data & Metrics
- [ ] Define a metrics framework for an iOS team (technical + product + team metrics)
- [ ] Design an A/B testing protocol for a mobile feature
- [ ] Create a mock executive dashboard for iOS team health
- [ ] Read "Accelerate" (Nicole Forsgren) — understand DORA metrics deeply

Deliverable: iOS Platform Strategy document + metrics framework ready to present to a hypothetical leadership team.

Month 6: Interview Preparation

Goal: Translate all your preparation into interview readiness.

Week 1-2: STAR Stories
- [ ] Write 20 STAR stories across all 5 behavioral themes
- [ ] Record yourself telling 5 of them — review for clarity, conciseness (aim for 3-4 min each)
- [ ] Get feedback from a peer manager or coach

Week 3: System Design
- [ ] Practice 8 mobile system design problems (use the templates in this guide)
- [ ] Present 3 designs to someone and get feedback
- [ ] Time yourself: requirements (5 min), high-level (10 min), deep dive (20 min), trade-offs (5 min)

Week 4: Mock Interviews
- [ ] Complete 3 mock technical iOS interviews (use LeetCode iOS problems or real iOS challenges)
- [ ] Complete 5 mock behavioral interviews (use the question bank)
- [ ] Complete 2 full mock EM interview loops (behavioral + system design + technical)

Deliverable: Interview-ready with prepared answers for every question in the question bank.

Daily Habits for iOS EM Candidates

30 Minutes/Day

Day	Focus
Monday	Read iOS Dev Weekly newsletter — stay current
Tuesday	Coding: Swift problem or iOS feature
Wednesday	STAR story practice (write or record one)
Thursday	Management reading (books, blogs, podcasts)
Friday	System design problem (15 min) + reflection
Weekend	Project work or deeper reading

Weekly

- Review WWDC sessions (1 session/week minimum)
- Follow 5 iOS engineering managers on LinkedIn/Twitter — read their posts
- Post one insight to LinkedIn or Twitter — practice communicating ideas publicly
- Connect with one person in iOS or EM community

Monthly

- Attend one virtual iOS meetup or conference

- Read one management book chapter
- Write one technical or leadership article (even if unpublished)
- Review and update your STAR story bank

Building Your Portfolio

GitHub Portfolio Projects

Project 1: Architecture Showcase
- MVVM+Coordinator app with real API
- 80%+ test coverage
- CI/CD pipeline
- README documenting architecture decisions

Project 2: Offline-First App
- Core Data or SwiftData
- Sync mechanism
- Conflict resolution strategy documented
- Performance metrics in README

Project 3: iOS SDK
- Clean public API
- Zero-dependency
- Documentation
- Example app

Project 4: Open Source Contribution
- Contribute to a real iOS open-source project
- Even documentation or bug fixes count
- Shows collaboration, code quality in public

Writing Portfolio

Publish on Medium, LinkedIn, or your own blog:

1. "How we migrated our iOS app to MVVM+Coordinator"
2. "Lessons from optimizing iOS app launch time by 40%"
3. "How I structure 1:1s as an iOS Engineering Manager"
4. "Building a modular iOS architecture for scale"
5. "What I wish I knew before becoming an Engineering Manager"

Resources: Priority Reading Order

If You Have 1 Month

1. "The Manager's Path" — Camille Fournier (management)
2. "Radical Candor" — Kim Scott (feedback)
3. WWDC: Swift Concurrency sessions (technical)
4. Stanford CS193p: SwiftUI (if SwiftUI gaps)

If You Have 3 Months

Add:
5. "High Output Management" — Andy Grove
6. "An Elegant Puzzle" — Will Larson
7. "Advanced iOS App Architecture" — Ray Wenderlich
8. Point-Free: TCA episodes
9. "Accelerate" — Nicole Forsgren

If You Have 6 Months

Add everything above plus:
10. "iOS Unit Testing by Example" — Jon Reid
11. "Designing Data-Intensive Applications" — Martin Kleppmann (backend awareness)
12. "Clean Architecture" — Robert C. Martin
13. "The Five Dysfunctions of a Team" — Patrick Lencioni
14. "Drive" — Daniel Pink
15. "Crucial Conversations" — Patterson et al.

Tracking Your Progress

Weekly Check-In Template

```
## Week [N] Progress Check

### iOS Technical
- What I practiced:
- Key insight gained:
- Gap still remaining:

### Management Skills
- What I practiced:
- Key insight gained:
- Gap still remaining:

### Interview Prep
- STAR stories written this week:
- System designs practiced:
- Mock interviews done:

### Next Week Priority
1.
2.
3.
```

Monthly Milestone Review

Score yourself again on the self-assessment table at the start of each month. You should see improvement in your targeted areas. If not:
- Was the practice deliberate? (targeted toward weakness, with feedback)
- Was it enough? (30+ min/day)
- Do you need a different resource or approach?

Signs You're Ready to Apply

Technical:
- [ ] You can explain any iOS topic in the README at a solid level
- [ ] You've built 2-3 iOS projects demonstrating architectural thinking
- [ ] You can profile an app and identify performance issues confidently
- [ ] You can design a system design answer end-to-end in 40 minutes

Management:
- [ ] You have 20+ STAR stories prepared and practiced
- [ ] You can answer every question in the interview question bank
- [ ] You've completed 5+ mock behavioral interviews
- [ ] You have clear, quantified examples of team impact

Mindset:

- [ ] You can articulate "why management" authentically and convincingly
- [ ] You understand the company's iOS products deeply
- [ ] You have genuine enthusiasm for helping others grow
- [ ] You can discuss your management philosophy coherently